PATITENT CONDITION AWARE DRUG RECOMMENDATION AND CLINICAL REVIEW SENTIMENT ANALYSIS USING NLP AND MACHINE LEARNING

A Project report submitted to

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING WITH DATA SCIENCE

In partial fulfilment of the requirements for the award of the Degree of

BACHELOR OF TECHNOLOGY IN

COMPUTER SCIENCE AND ENGINEERING WITH DATA SCIENCE

CHALAPATHI INSTITUTE OF ENGINEERING & TECHNOLOGY

By

RAVULA NARENDRA	(Y22CDS045)
EDA SAMATHA	(Y22CDS012)
RAVULAKOLLU MANI KUMAR	(Y22CDS046)
SHAIK SAMEER	(Y22CDS051)
VARRA VIVEK VARDHAN REDDY	(Y22CDS058)

Under the Guidance of

Mr. S. VENKATA. DURGA PRASAD, M. Tech

ASSISTANT PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING WITH DATA SCIENCE

CHALAPATHI INSTITUTE OF ENGINEERING AND TECHNOLOGY (AUTONOMOUS)

(Accredited by NAAC with 'A' grade, accredited by NBA, Approved by A.I.C.T.E, Affiliated To Acharya Nagarjuna University)

GUNTUR-522034

2025-2026

CHALAPATHI INSTITUTE OF ENGINEERING AND TECHNOLOGY (AUTONOMOUS)

(Accredited by NAAC with 'A' grade, accredited by NBA, Approved by A.I.C.T.E, Affiliated to Acharya Nagarjuna University)

CHALAPATHI NAGAR, LAM, GUNTUR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING WITH DATA SCIENCE

CHALAPATHI INSTITUTE OF ENGINEERING & TECHNOLOGY

CERTIFICATE

This is to certify that the project work entitled as "***Patient Condition-Aware Drug Recommendation and Clinical Review Sentiment Analysis Using Machine Learning and NLP***" submitted by **Ravula Narendra(Y22CDS045), Eda Samatha(Y22CDS012), Ravulakollu Mani Kumar(Y22CDS046), Shaik Sameer(Y22CDS051)** and **Varra Vivek Vardhan(Y22CDS058)** in partial fulfilment for the award of the Degree of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING WITH DATA SCIENCE** is a record of bona fide work carried out under my guidance and supervision.

GUIDE	HEAD OF THE DEPARTMENT	PRINCIPAL
Mr. S. VENKATA DURGA PRASAD M. Tech ASSISTANT PROFESSOR	Mrs. K. ARUNA KUMARI M. Tech, (Ph. D) CSDS-HOD	Dr. M. CHANDRA SEKHAR M. Tech, (Ph. D)

ACKNOWLEDGEMENT

We express our sincere thanks to our beloved Chairman sir, **Sri. Y. V. ANJANEYULU** for providing support and stimulating environment for developing the project.

We express deep sense of reverence and profound gratitude to **Dr. M. CHANDRA SEKHAR, Ph. D,** Principal for providing us the great support in carrying out the project.

It plunges us in exhilaration in taking privilege in expressing our heartfelt gratitude to **Mrs. K. ARUNA KUMARI**, **M. Tech**, **(Ph. D),** HOD-CSDS for providing us every facility and for constant supervision.

We are thankful to our guide **S. VENKATA DURGA PRASAD, M. Tech,** Assistant professor, Dept. of CSDS for his constant encouragement, suggestions, supervision, and abundant support throughout the project.

Thanks to all the teaching and non-teaching staff and lab technicians for their support and also to our team mates for their valuable Co-operation.

By

RAVULA NARENDRA	**(Y22CDS045)**
EDA SAMATHA	**(Y22CDS012)**
RAVULAKOLLU MANI KUMAR	**(Y22CDS046)**
SHAIK SAMEER	**(Y22CDS051)**
VARRA VIVEK VARDHAN REDDY	**(Y22CDS058)**

CONTENTS

S.NO	CHAPTER	PAGENO
	Abstract & Problem statement	**I, II**
1	Introduction	**1**
2	Literature review	**4**
3	System Analysis & System Requirements	**8**
4	System Design	**12**
5	System Implementation	**24**
6	System Testing	**52**
7	Results and Discussion	**55**
8	Conclusion	**59**
9	Reference	**62**

INDEX

TABLE OF CONTENTS

LIST OF FIGURES

ABBREVIATIONS **I**

ABSTRACT **II**

PROBLEM STATEMENT **III**

CHAPTER – 1 INTRODUCTION **1-3**

1.1 Background of Intelligent Healthcare System 1

1.2 Importance of Personalized Drug Recommendation 2

1.3 Role of Clinical Reviews in Healthcare Analytics 2

1.4 Natural Language Processing in Healthcare 2-3

1.5 Objectives of the Project 3

1.6 Scope of the Project 3

1.7 Significance of the Study 3

CHAPTER - 2 LITERATURE REVIEW **4-7**

2.1 Evolution of Drug Recommendation Systems 4

2.2 Personalized Medicine and Machine Learning 4-5

2.3 Sentiment Analysis in Healthcare 5

2.4 Natural Language Processing Techniques in Medical Text Mining 5-6

2.5 Supervised Learning Algorithms in Healthcare Prediction 6

2.6 Integration of Structured and Unstructured Data 6-7

2.7 Evaluation Metrics in Healthcare ML Systems 7

2.8 Research Gap and Contribution 7

CHAPTER- 3 SYSTEM ANALYSIS & SYSTEM REQUIREMENTS **8-11**

3.1 Existing System Analysis 8

3.2 Proposed System Analysis 8-9

3.3 Feasibility Analysis 9

3.4 Functional Requirements 9-10

3.5 Non-Functional Requirements 10

3.6 Hardware Requirements 10

3.7 Software Requirements 11

3.8 System Constraints and Assumptions 11

CHAPTER – 4 SYSTEM DESIGN 12-23

4.1 Overall System Architecture 12-14

4.2 Data Input Module Design 14-15

4.3 Data Preprocessing Module Design 15

4.4 Drug Recommendation Model Design 15

4.5 Clinical Review Sentiment Analysis Design 15-16

4.6 Integration and Ranking Module Design 16

4.7 Output Interface Design 16

4.8 System Workflow Design 16-18

4.9 UML Diagrams 18-23

4.9.1 Use Case Diagram 18-19

4.9.2 Class Diagram 20

4.9.3 Sequence Diagram 21

4.9.4 Activity Diagram 22

4.9.5 Component Diagram 23

CHAPTER- 5 SYSTEM IMPLEMENTATION 24-51

5.1 Development Environment Setup 24-34

5.1.1 Technologies Used 27-28

5.1.2 How to Install Python on Windows and Mac 29-31

5.1.3 Installation of Python 31-32

5.1.4 Verify the Python Installation 32-34

5.2 Structured Data Loading and Exploration 34

5.3 Text Data Loading and Preprocessing 35-36

5.4 Drug Recommendation Model Training 36

5.5 Sentiment Classification Model Training 36-37

5.6 Sentiment Score Aggregation 37

5.7 Integration of Recommendation and Sentiment Modules 37

5.8 System Deployment and Testing 38-41

5.9 Source Code: 42-51

CHAPTER-6 SYSTEM TESTING **52-54**

6.1 Testing Strategy 52

6.2 Unit Testing of Drug Recommendation Module 52

6.3 Unit Testing of Sentiment Analysis Module 53

6.4 Integration Testing 53

6.5 System Testing 53

6.6 Performance Testing 54

6.7 Accuracy and Validation Testing 54

6.8 Limitations Identified During Testing 54

CHAPTER - 7 RESULTS AND DISCUSSION **55-59**

7.1 Overall System Performance 55

7.2 Drug Recommendation Model Results 55

7.3 Sentiment Analysis Results 56

7.4 Impact of Sentiment Integration on Drug Ranking 56

7.5 Comparative Analysis with Traditional Systems 56

7.6 User Interpretation and Transparency 57

7.6 Limitations and Observed Challenges 57

7.7 Future Scope and Overall Discussion 57-58

CHAPTER - 8 CONCLUSION **59-61**

REFERENCES **62-64**

LIST OF FIGURES

S. No	Figure. No	CONTENT	Page. No
1	Fig: 4.1.1	System Architecture of Proposed Drug Recommendation System	13
2	Fig: 4.8.1	System Design	17
3	Fig: 4.9.1	Use Case Diagram	19
4	Fig: 4.9.2	Class Diagram	20
5	Fig: 4.9.3	Sequence Diagram	21
6	Fig: 4.9.4	Activity Diagram	22
7	Fig: 4.9.5	Component Diagram	23
8	Fig: 5.1.1	Requirements	24
9	Fig: 5.1.2	Package install	25
10	Fig: 5.3.1	NLP-Based Text Data Loading and Preprocessing Workflow	35
11	Fig: 5.8.1	Folder Structure	38
12	Fig: 5.8.2	Libraries	39
13	Fig: 5.8.3	Environment Setup	40
14	Fig: 5.8.4	Interface	41
15	Fig:5.9.1	Output 1	48
16	Fig: 5.9.2	Output 2	50

ABBREVATIONS

- **NLP** – Natural Language Processing
- **ML** – Machine Learning
- **BMI** – Body Mass Index
- **EHR** – Electronic Health Records
- **TF-IDF** – Term Frequency–Inverse Document Frequency
- **SVM** – Support Vector Machine(s)
- **LSTM** – Long Short-Term Memory
- **EDA** – Exploratory Data Analysis
- **API** – Application Programming Interface
- **CSV** – Comma-Separated Values
- **CPU** – Central Processing Unit
- **RAM** – Random Access Memory
- **AI** – Artificial Intelligence
- **WHO** – World Health Organization

ABSTRACT

Healthcare decision-making is increasingly driven by data to improve treatment accuracy and patient safety. With the rapid growth of electronic health records, online medical reviews, and pharmaceutical databases, there is a need for intelligent systems that can analyze patient conditions and recommend appropriate medications while also evaluating public sentiment toward drugs. Traditional drug recommendation systems often rely on static rule-based approaches or physician experience, lacking personalization and contextual awareness. Additionally, patient reviews and clinical feedback are often underutilized despite offering valuable insights into drug effectiveness and side effects.

This project, titled "**Patient Condition–Aware Drug Recommendation and Clinical Review Sentiment Analysis Using NLP and Machine Learning**," proposes an integrated framework that combines condition-based drug recommendation with sentiment analysis of clinical reviews. The system uses patient data such as diagnosis, symptoms, age, and medical history to recommend suitable medications using supervised machine learning algorithms, while Natural Language Processing (NLP) techniques analyze patient reviews to classify sentiment as positive, negative, or neutral.

The recommendation engine applies classification or ranking algorithms to match patient conditions with appropriate drugs from a structured pharmaceutical dataset. Text preprocessing techniques such as tokenization, stop word removal, stemming, and TF-IDF vectorization are applied before training models like Logistic Regression or Support Vector Machines for sentiment classification. Results show that combining structured clinical data with unstructured review analysis improves recommendation reliability. This approach enhances decision support, increases transparency, and contributes to more patient-centered healthcare analytics.

KEYWORDS: *Drug Recommendation, Patient Condition Analysis, Machine Learning, Natural Language Processing, Sentiment Analysis, Healthcare Analytics*

PROBLEM STATEMENT

Existing drug recommendation systems primarily rely on structured medical guidelines and ignore real-world patient feedback. Additionally, patient reviews remain underutilized in clinical decision support systems. The problem addressed in this project is to develop a patient condition-aware drug recommendation system that integrates machine learning-based classification with NLP driven sentiment analysis of clinical reviews. The system must recommend drugs based on patient attributes while evaluating public sentiment to enhance reliability.

CHAPTER – 1

INTRODUCTION

1.1 Background of Intelligent Healthcare System

The healthcare industry has witnessed significant digital transformation in recent years. Electronic Health Records (EHRs), clinical databases, wearable health monitors, and online medical forums generate vast amounts of structured and unstructured data. Leveraging this data effectively can improve diagnosis accuracy, treatment personalization, and patient outcomes. Machine learning and Natural Language Processing (NLP) have emerged as powerful tools for analyzing healthcare data and supporting clinical decision-making.

Drug recommendation systems play a crucial role in assisting physicians by suggesting suitable medications based on patient conditions. Traditional systems rely heavily on predefined clinical guidelines and physician expertise. While these approaches are reliable, they may not incorporate patient-specific contextual data or real-world feedback from other patients

Simultaneously, online medical review platforms provide valuable insights into drug effectiveness and side effects. However, extracting meaningful information from textual reviews requires advanced NLP techniques.

This project integrates structured patient condition data with unstructured review sentiment analysis to build a comprehensive, data-driven drug recommendation framework. By combining these components, the system enhances decision support and patient-centric healthcare analytics.

1.2 Importance of Personalized Drug Recommendation

Personalized medicine aims to tailor medical treatment to individual patient characteristics. Factors such as age, gender, pre-existing conditions, allergies, and symptom severity influence drug effectiveness and safety. A generic recommendation may not produce optimal results for every patient.

Machine learning models can analyze patterns in historical clinical data to predict suitable medications for specific conditions. Classification algorithms can map patient attributes to drug labels, improving treatment accuracy.

Personalized drug recommendation reduces adverse drug reactions and enhances therapeutic outcomes. It supports clinicians by providing evidence-based suggestions derived from large datasets.

This project focuses on building a patient condition-aware recommendation model that considers multiple patient parameters, improving personalization and safety in drug selection.

1.3 Role of Clinical Reviews in Healthcare Analytics

Patient reviews contain experiential information about drug effectiveness, dosage tolerance, and side effects. Unlike structured clinical trial data, reviews provide real-world insights from diverse patient populations.

However, these reviews are unstructured text data and require NLP techniques for analysis. Sentiment analysis can classify reviews into positive, negative, or neutral categories, revealing overall patient perception of a drug. Incorporating review sentiment into drug recommendation systems enhances transparency and trust. A drug with high positive sentiment and lower negative feedback may be prioritized.

This project uses NLP-based sentiment classification to extract meaningful patterns from clinical reviews and integrate them into the recommendation framework.

1.4 Natural Language Processing in Healthcare

NLP enables machines to understand and process human language. In healthcare, NLP is used for medical record analysis, disease classification, and review mining. Text preprocessing steps include tokenization, stop word removal, stemming, and vectorization. Machine learning classifiers such as Logistic Regression, Naïve Bayes, and Support Vector Machines are commonly applied for sentiment analysis. Advanced NLP

techniques improve accuracy in extracting healthcare insights from unstructured data.

1.5 Objectives of the Project

The primary objectives include:

1.5.1 Developing a condition-based drug recommendation model.

1.5.2 Implementing NLP-based sentiment analysis for clinical reviews.

1.5.3 Integrating structured and unstructured data sources.

1.5.4 Evaluating model performance using accuracy, precision, recall, and F1-score.

1.5.5 Enhancing personalized healthcare decision support.

1.6 Scope of the Project

The project focuses on supervised machine learning for drug recommendation and text classification for review sentiment analysis. It does not include deep learning-based medical imaging or genomic analysis.

The system can be integrated into healthcare platforms or hospital decision support systems. Future scope may include multi-label classification and adverse drug reaction prediction.

1.7 Significance of the Study

This study demonstrates the integration of structured clinical data and unstructured review analysis in healthcare recommendation systems. It enhances personalization, transparency, and reliability in drug selection.

CHAPTER - 2

LITERATUREREVIEW

2.1 Evolution of Drug Recommendation Systems

Drug recommendation systems have evolved from traditional rule-based expert systems to advanced machine learning-driven decision support tools. Early systems relied heavily on predefined clinical guidelines, pharmacological rules, and physician expertise. While these systems provided structured recommendations, they lacked adaptability and personalization. With the growth of Electronic Health Records (EHRs) and digital medical databases, machine learning models began to assist in treatment prediction. Classification algorithms such as Decision Trees, Random Forests, and Support Vector Machines were applied to patient data to recommend medications based on diagnosis and symptoms.

Recent research emphasizes data-driven clinical decision support systems that analyze large healthcare datasets to identify patterns and optimize treatment outcomes. However, many existing systems focus solely on structured clinical data and overlook patient-generated content such as online reviews and feedback.

The integration of patient reviews into drug recommendation remains relatively underexplored. Combining structured medical data with unstructured textual analysis offers a more holistic approach to drug evaluation.

This project builds upon machine learning-based drug recommendation research and extends it by incorporating NLP-based sentiment analysis for clinical review evaluation.

2.2 Personalized Medicine and Machine Learning

Personalized medicine aims to tailor treatment strategies to individual patient characteristics. Traditional one-size-fits-all approaches may not yield optimal outcomes due to variations in age, genetic profile, lifestyle, and comorbidities.

Machine learning plays a crucial role in personalized medicine by identifying patterns in patient data. Supervised learning algorithms can classify patient conditions and predict suitable medications based on historical records.

Research indicates that predictive models can reduce adverse drug reactions and improve therapeutic efficiency. For example, classification models trained on patient features can predict the most effective drug for specific conditions.

The proposed system addresses this gap by integrating patient condition-aware machine learning models with sentiment analysis of drug reviews to enhance personalization.

2.3 Sentiment Analysis in Healthcare

Sentiment analysis, a subfield of Natural Language Processing (NLP), is widely used to classify opinions expressed in text. In healthcare, sentiment analysis has been applied to patient feedback, online forums, and clinical reviews.

Studies show that sentiment classification helps identify drug effectiveness, side effects, and patient satisfaction levels. Positive sentiment often correlates with perceived treatment success, while negative sentiment highlights adverse reactions or inefficacy.

Common algorithms used for sentiment analysis include Naïve Bayes, Logistic Regression, Support Vector Machines, and deep learning models such as LSTM networks. Feature extraction techniques such as Bag-of-Words and TF-IDF are widely used for text representation.

Healthcare sentiment analysis faces challenges such as domain-specific terminology, sarcasm, and complex medical expressions. Despite these challenges, research demonstrates that NLP can effectively extract meaningful insights from clinical reviews.

This project leverages sentiment analysis to evaluate drug reviews and integrate sentiment scores into the recommendation framework.

2.4 Natural Language Processing Techniques in Medical Text Mining

Medical text mining involves extracting structured information from unstructured clinical text. NLP techniques such as tokenization, stemming, lemmatization, and stopword removal are essential preprocessing steps.

Vectorization methods like TF-IDF and word embeddings convert textual data into numerical representations for machine learning models. Domain-specific lexicons improve classification accuracy in medical contexts.

Research highlights the importance of preprocessing quality in improving sentiment classification performance. Handling medical abbreviations and specialized terminology is critical for accurate results.

Advanced models such as BERT have been applied in clinical NLP tasks, achieving higher contextual understanding. However, traditional machine learning models remain effective and computationally efficient for many applications.

The proposed system utilizes NLP preprocessing and TF-IDF vectorization for sentiment classification of drug reviews.

2.5 Supervised Learning Algorithms in Healthcare Prediction

Supervised learning algorithms are widely used for healthcare prediction tasks such as disease classification, drug effectiveness prediction, and risk assessment. Logistic Regression, Random Forest, Decision Trees, and Support Vector Machines are commonly applied.

Logistic Regression is effective for binary classification tasks such as sentiment classification. Random Forest provides robustness and handles nonlinear relationships effectively.

Research indicates that ensemble methods improve predictive accuracy and reduce overfitting in healthcare datasets. Performance metrics such as accuracy, precision, recall, and F1-score are used for evaluation.

The proposed system employs supervised learning algorithms for both drug recommendation classification and sentiment analysis tasks.

2.6 Integration of Structured and Unstructured Data

Healthcare data exists in both structured formats (EHRs, lab reports) and unstructured formats (clinical notes, patient reviews). Integrating these data types enhances decision-making. Studies show that combining structured clinical parameters with textual sentiment analysis improves predictive performance in medical recommendation systems. However, integrating heterogeneous data requires careful preprocessing and

normalization. Feature fusion techniques enable the combination of structured attributes and textual sentiment scores. The proposed system integrates patient condition attributes with sentiment analysis results to create a comprehensive recommendation framework.

2.7 Evaluation Metrics in Healthcare ML Systems

Healthcare machine learning systems require rigorous evaluation to ensure reliability. Classification models are evaluated using accuracy, precision, recall, F1-score, and confusion matrices.

In drug recommendation systems, high recall is important to minimize missed effective treatments, while high precision reduces incorrect recommendations.

Sentiment classification models require balanced evaluation to avoid bias toward dominant classes. Cross-validation techniques enhance model generalization and robustness. The proposed system employs these evaluation metrics to validate performance in both recommendation and sentiment analysis modules.

2.8 Research Gap and Contribution

Although machine learning-based drug recommendation systems and healthcare sentiment analysis models exist independently, limited research integrates both components into a unified framework. Most existing systems either focus on clinical guideline-based recommendation or analyze reviews separately. The research gap lies in combining patient condition-aware prediction with sentiment- driven drug evaluation.

This project contributes by:

- Developing a condition-aware drug recommendation model.
- Implementing NLP-based sentiment analysis of clinical reviews.
- Integrating structured and unstructured data sources.
- Enhancing personalization and transparency in healthcare decision support.

The study advances intelligent healthcare analytics by bridging machine learning and NLP in drug recommendation systems.

CHAPTER - 3

SYSTEMANALYSIS & SYSTEM REQUIREMENTS

3.1 Existing System Analysis

Existing drug recommendation systems primarily rely on clinical guidelines, rule-based expert systems, or physician experience. These systems use structured patient data such as diagnosis, lab reports, and medical history to determine appropriate medications. While such systems provide medically sound recommendations, they often lack personalization beyond clinical parameters and do not incorporate real world patient experiences.

Another limitation is the separation between structured and unstructured healthcare data. Electronic Health Records (EHRs) provide structured data, whereas patient reviews, feedback forums, and clinical comments are unstructured text. Most traditional systems ignore unstructured textual data, thereby missing valuable insights regarding drug effectiveness, side effects, and patient satisfaction.

Additionally, existing review analysis systems operate independently from recommendation engines. They may classify sentiment but do not influence drug ranking decisions. This separation reduces the potential impact of real-world feedback on treatment decisions.

Furthermore, static rule-based systems lack adaptability and cannot continuously learn from new patient data. Machine learning-based healthcare systems are emerging but often focus solely on prediction accuracy without considering transparency or patient perception.

These limitations highlight the need for an integrated, data-driven system that combines patient condition aware drug recommendation with NLP-based sentiment analysis to enhance reliability and personalization.

3.2 Proposed System Analysis

The proposed system integrates two primary components: a patient condition-aware drug recommendation model and a clinical review sentiment analysis module. The system is designed to process both structured patient data and unstructured review text data.

In the first phase, patient attributes such as diagnosis, symptoms, age, and medical history are input into a supervised machine learning model. This model predicts suitable medications based on historical clinical data patterns.

In the second phase, patient reviews for recommended drugs are analyzed using NLP techniques. Sentiment classification identifies whether reviews are positive, negative, or neutral. Sentiment scores are then incorporated into the drug ranking process.

The system combines predictive modeling with textual sentiment insights to generate context- aware and transparent recommendations. Drugs with high predicted suitability and favorable sentiment scores are prioritized.

This integrated approach enhances personalization, improves trustworthiness, and supports clinical decision-making by providing both clinical and experiential insights.

3.3 Feasibility Analysis

The feasibility of the proposed system is evaluated in terms of technical, economic, and operational feasibility.

From a technical perspective, required tools such as Python, Scikit-learn, and NLP libraries are widely available. Machine learning algorithms for classification and sentiment analysis are computationally efficient and suitable for moderate hardware configurations.

Economically, the system is cost-effective as it relies on open-source software and publicly available healthcare datasets. No specialized hardware is required unless integrated into hospital systems.

Operational feasibility is high because healthcare providers can use the system as a decision- support tool rather than a replacement for medical expertise. The system enhances rather than overrides physician decisions.

Overall, the system is practical, scalable, and suitable for integration into digital healthcare platforms.

3.4 Functional Requirements

Thefunctional requirements define what the system must accomplish.

1. The system must accept structured patient condition data.
2. The system must preprocess patient attributes for model training.
3. The system must recommend drugs based on classification or ranking algorithms.

4. The system must collect and preprocess clinical review text data.
5. Thesystem must perform sentiment analysis on reviews.
6. Thesystem must integrate sentiment scores into drug ranking.
7. The system must display recommended drugs along with sentiment insights.
8. Thesystem must allow model retraining with updated data.

These requirements ensure comprehensive functionality for both recommendation and sentiment analysis.

3.5 Non-Functional Requirements

Non-functional requirements ensure system quality and reliability.

Accuracy: The recommendation and sentiment models must achieve high predictive accuracy.

Performance: The system must generate recommendations within minimal response time.

Scalability: The system should handle increasing patient records and review data.

Security: Patient data must be securely stored and processed according to privacy regulations.

Reliability: The system must provide consistent recommendations under various conditions.

Maintainability: The architecture must allow easy updates and algorithm improvements. Usability: The interface must be simple for healthcare professionals. These attributes ensure robustness and practical deployment readiness.

3.6 Hardware Requirements

The hardware requirements are minimal. A standard computer with at least 4–8 GB RAM and a multicore processor is sufficient for development and testing.

For hospital-scale deployment, server infrastructure or cloud hosting maybe required to handle larger datasets.

No specialized medical devices are required unless integrated with real-time EHR systems. The

system's low hardware dependency ensures accessibility and scalability.

3.7 Software Requirements

The software requirements include:

- Python 3.x
- Pandas and NumPy for data manipulation
- Scikit-learn for classification algorithms
- NLTK or spaCy for NLP preprocessing
- TF-IDF Vectorizer for text representation
- Flask or Django for optional web interface
- Database systems such as MySQL or SQLite

These tools provide a comprehensive ecosystem for implementing and deploying the system.

3.8 System Constraints and Assumptions

The system assumes that patient data is accurate and complete. Incomplete or incorrect data may reduce recommendation reliability.

Sentiment analysis accuracy depends on the clarity of review text and proper preprocessing. The model does not replace professional medical advice; it functions as a decision-support system. Data privacy regulations must be followed when handling patient information.

CHAPTER - 4

SYSTEMDESIGN

4.1 Overall System Architecture

The overall system architecture follows a **modular and layered design approach**, ensuring scalability, maintainability, and efficient data processing across different components. The system is divided into six primary modules: **Data Input Module, Data Preprocessing Module, Drug Recommendation Module, Clinical Review Sentiment Analysis Module, Integration & Ranking Module, and Output Interface Module**. Each module performs a specific function while maintaining seamless interaction with other components, thereby enabling a robust and flexible healthcare decision-support system.

The process begins with the **Data Input Module**, which captures structured patient information such as age, gender, diagnosis, symptoms, medical history, allergies, and current medications. In addition to structured inputs, the system also supports unstructured data in the form of textual descriptions or uploaded clinical reports. This flexibility allows the system to handle real-world scenarios where patient data may not always be in a standardized format.

Once the data is collected, it is passed to the **Data Preprocessing Module**, where it undergoes cleaning, normalization, and transformation. Missing values are handled, categorical variables are encoded, and numerical features are scaled appropriately. For unstructured text data, natural language preprocessing techniques such as tokenization, stop-word removal, and feature extraction are applied. This step ensures that the data is in a suitable format for machine learning models, thereby improving prediction accuracy and system reliability.

The processed data is then fed into the **Drug Recommendation Module**, which uses supervised machine learning algorithms to analyze patient conditions and predict suitable medications. This module considers multiple factors, including patient-specific attributes and clinical guidelines, to generate a list of recommended drugs. The model outputs not only the predicted drugs but also confidence scores that indicate the reliability of each recommendation.

Simultaneously, the **Clinical Review Sentiment Analysis Module** processes unstructured textual reviews related to the recommended drugs. Using advanced Natural Language Processing (NLP) techniques and pre-trained models, this module classifies reviews into

sentiment categories such as positive, negative, or neutral. The outputs from both modules are then combined in the **Integration & Ranking Module**.

This module plays a critical role in enhancing decision-making by merging clinical predictions with sentiment insights. It applies ranking algorithms to prioritize drugs that are not only clinically appropriate but also positively perceived by patients. This dual evaluation ensures a more holistic and patient-centric recommendation system.

Finally, the **Output Interface Module** presents the results to the user through an interactive and user-friendly interface. It displays ranked drug recommendations along with confidence scores, sentiment summaries, and detailed insights. Visualization tools such as tables and charts may be used to improve interpretability. Additionally, the interface may include disclaimers and explanations to assist users in understanding the recommendations.

This architecture ensures a clear separation of concerns, allowing each module to be developed, tested, and updated independently. At the same time, the integration between modules enables smooth data flow and real-time processing. Overall, the system provides a comprehensive framework that combines structured clinical data with unstructured patient feedback, resulting in more accurate, transparent, and reliable healthcare recommendations.

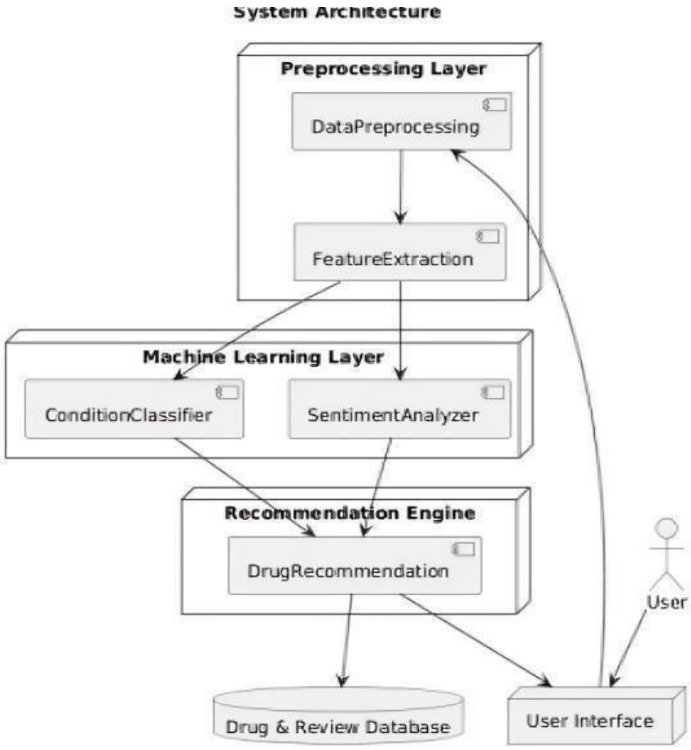


Fig : System Architecture of Proposed Drug Recommendation System

Fig:4.1.1-System Architecture

The diagram illustrates the **system architecture of a patient condition–aware drug recommendation system**. It is organized into multiple layers that work sequentially to process data and generate recommendations.

At the top is the **Preprocessing Layer**, which includes *Data Preprocessing* and *Feature Extraction*. In this stage, raw input data such as patient conditions and drug reviews are cleaned, normalized, and transformed into meaningful features that can be used by machine learning models.

The processed data is then passed to the **Machine Learning Layer**, which consists of two main components: *Condition Classifier* and *Sentiment Analyzer*. The condition classifier identifies the patient's medical condition based on input data, while the sentiment analyzer evaluates user reviews to determine whether the feedback on drugs is positive, negative, or neutral.

Next, the outputs from these models are fed into the **Recommendation Engine**, where the *Drug Recommendation* module generates suitable drug suggestions by combining predicted conditions with sentiment insights from reviews.

The system also interacts with a **Drug & Review Database**, which stores historical drug information and user feedback, enabling more accurate recommendations. Finally, the **User Interface** allows users to input their data and view recommended drugs. The feedback loop from the user interface back to preprocessing indicates continuous data flow and system improvement. Overall, the architecture demonstrates an integrated approach combining data preprocessing, machine learning, and recommendation techniques to support intelligent healthcare decision- making.

4.2 Data Input Module Design

The Data Input Module collects structured patient condition data and unstructured drug review text data. Patient data includes:

- Age
- Gender
- Diagnosis
- Symptoms
- Pre-existing conditions
- Allergy information

This structured data is stored in tabular format for easy processing.

Drug review data is collected from healthcare forums or public medical review datasets. Reviews are stored as textual data linked to specific drug names. Input validation mechanisms ensure that missing or invalid entries are handled properly. For example, age must be within a realistic range, and diagnosis fields must not be empty.

4.3 Data Preprocessing Module Design

The preprocessing module prepares structured and unstructured data for machine learning tasks. For structured patient data:

- Missing values are handled using imputation techniques.
- Categorical variables such as gender or diagnosis are encoded using label encoding or one- hot encoding.
- Numerical features are normalized to ensure consistent scaling. For textual review data:
- Tokenization is performed to split text into words.
- Stopwords are removed.
- Stemmingor lemmatization is applied.
- Text is converted into numerical vectors using TF-IDF.

This preprocessing ensures clean, consistent, and structured data ready for model training and prediction.

4.4 Drug Recommendation Model Design

The Drug Recommendation Module uses supervised classification algorithms to

predict appropriate medications based on patient attributes. The model is trained on historical clinical datasets containing patient conditions and prescribed drugs. The objective is to map patient feature vectors to drug labels. Cross-validation techniques ensure robust generalization.

4.5 Clinical Review Sentiment Analysis Design

The Sentiment Analysis Module processes drug review text to determine overall sentiment. After preprocessing, TF-IDF vectorization converts reviews into numerical representations. Classification algorithms such as Logistic Regression or Naïve Bayes are

used to categorize reviews as positive, negative, or neutral.

The system aggregates sentiment scores for each drug. For example, a drug with 70% positive reviews receives a higher sentiment score. Confusion matrix analysis ensures balanced performance across classes.

This module extracts real-world patient insights and provides additional evaluation metrics for recommended drugs.

4.6 Integration and Ranking Module Design

The Integration Module combines outputs from the recommendation and sentiment modules. Each recommended drug is assigned:

- Clinical prediction score
- Sentiment score

A weighted ranking formula integrates both scores to produce a final ranking. For example:

Final Score = (0.7 × Prediction Score) + (0.3 × Sentiment Score Weights can be adjusted depending on system priorities.

This design ensures that drugs with strong clinical suitability and positive patient sentiment are prioritized.

4.7 Output Interface Design

The Output Interface presents recommendations in a clear and interpretable format.

Displayed information includes:

- Drug name
- Predicted suitability score
- Sentiment distribution (positive/negative percentage)
- Confidence score

Visual elements such as bar charts or rating indicators maybe included. The interface is designed for healthcare professionals, ensuring clarity and ease of interpretation.

4.8 System Workflow Design

The system workflow follows these steps:

1. Patient data input
2. Structured data preprocessing

3. Drug prediction using ML model
4. Review text preprocessing
5. Sentiment classification
6. Sentiment aggregation
7. Scoreintegration and ranking
8. Display of final recommendations

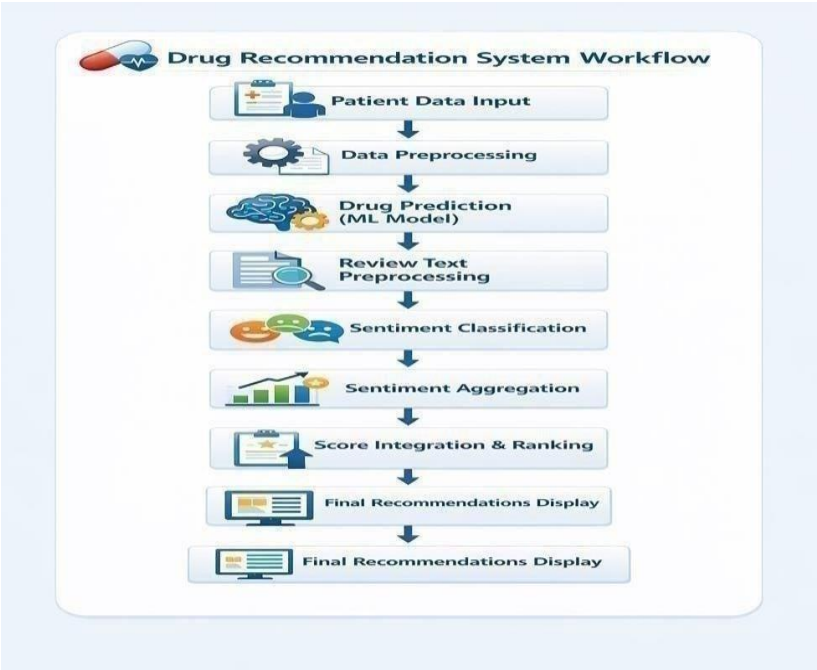


Fig:4.8.1-System Design

The diagram shows the **workflow of a drug recommendation system**, illustrating how patient data is processed step-by-step to generate accurate drug suggestions. The process begins with **Patient Data Input**, where the user provides information such as symptoms, medical conditions, or health history. This input is then passed to the **Data Preprocessing** stage, where the data is cleaned, structured, and prepared for analysis.

Next, the system performs **Drug Prediction using a Machine Learning (ML) Model**, which predicts potential drugs based on the processed patient data and learned patterns from historical datasets.

In parallel, the system also analyzes user-generated drug reviews. The **Review Text Preprocessing** stage cleans and formats textual review data, which is then fed into **Sentiment Classification**. Here, reviews are categorized as positive, negative, or neutral to understand user opinions about different drugs.

After classification, the results are combined in the **Sentiment Aggregation** step, where overall sentiment scores for each drug are calculated. These scores reflect the general effectiveness and user satisfaction levels.

The system then performs **Score Integration & Ranking**, where it combines the ML-based drug predictions with sentiment scores to rank the most suitable drugs.

Finally, the **Final Recommendations Display** presents the top-ranked drug suggestions to the user through the interface.

Overall, the workflow highlights how the system integrates machine learning predictions with sentiment analysis to provide more reliable and user-informed drug recommendations.

4.9 UML DIAGRAMS

UML diagrams (Unified Modeling Language diagrams) are visual representations used to design, understand, and document a system, especially in software development.

They help you show how a system works, how different parts interact, and how it is structured—using standard symbols and diagrams.

4.9.1 Use Case Diagrams

The **Use Case Diagram** represents the interaction between the user and the *Patient Condition-Aware Drug Recommendation System*. It defines the functional requirements of the system from the user's perspective.

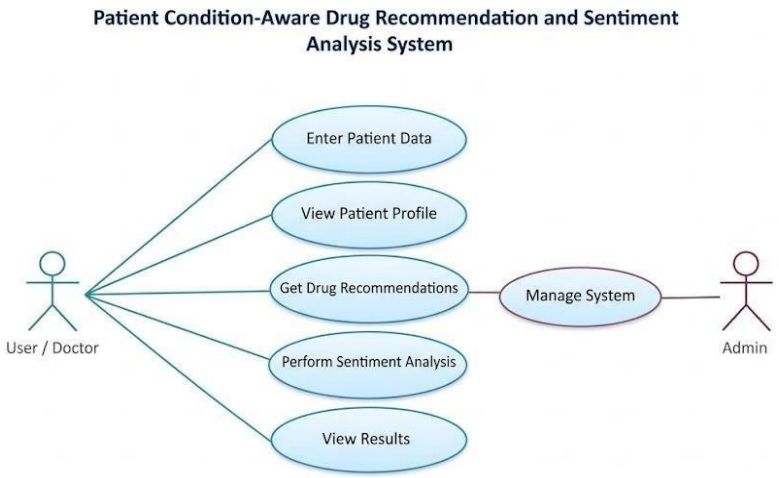
In this system, the **Patient** is the primary actor who interacts with the system to receive personalized drug recommendations.

The major use cases include:

- **Enter Symptoms:**
  The patient provides details about symptoms, medical history, and existing conditions.
  This input is essential for accurate prediction and recommendation.

- **Analyze Condition:**
  The system processes the entered symptoms using machine learning algorithms to identify possible medical conditions.
- **Get Drug Recommendation:**
  Based on the predicted condition, the system suggests suitable drugs using trained ML models.
- **Provide Review:**
  The user can give feedback or reviews about the recommended drugs.
- **Sentiment Analysis:**
  The system analyzes user reviews using NLP techniques to determine whether feedback is positive, negative, or neutral.
- **View Results:**
  The patient views the final recommended drugs along with sentiment-based filtering and safety considerations.

This diagram highlights that the system is **user-driven**, and all operations are initiated through user interaction.



4.9.1 - Use Case Diagram

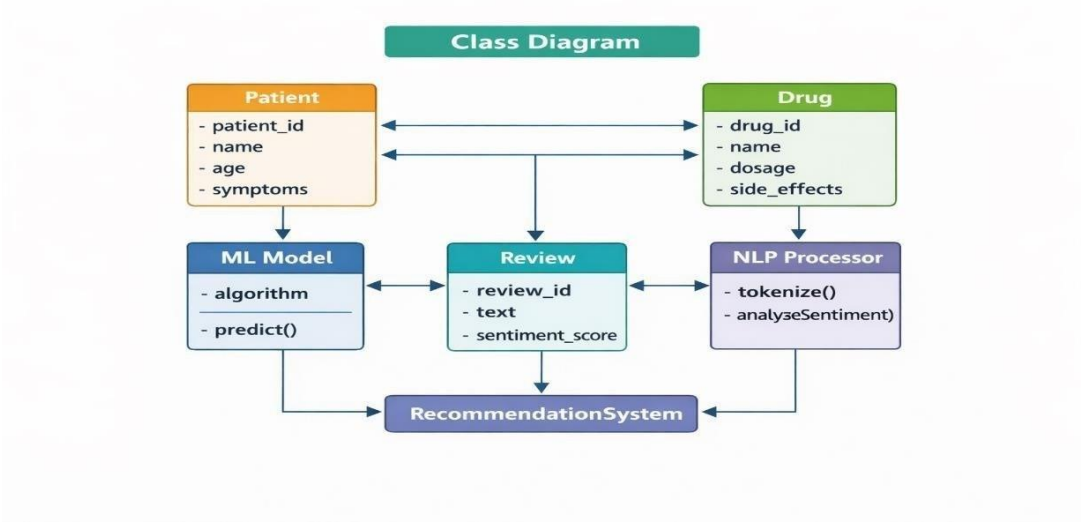
4.9.2 Class Diagram

The **Class Diagram** represents the static structure of the system. It shows the classes, their attributes, methods, and relationships.

In this system, the main classes include:

- **Patient:**
  Stores patient information such as ID, name, age, and symptoms. It acts as the input provider.
- **Drug:**
  Contains drug-related details like drug name, dosage, and side effects.
- **Review:**
  Stores user feedback, including review text and sentiment score.
- **ML Model:**
  Responsible for predicting diseases and recommending drugs based on symptoms.
- **NLP Processor:**
  Performs text processing tasks such as tokenization and sentiment analysis.
- **Recommendation System:**
  Integrates ML and NLP modules to generate final recommendations.

The relationships between these classes ensure smooth data flow and system functionality.



4.9.2 - Class Diagram

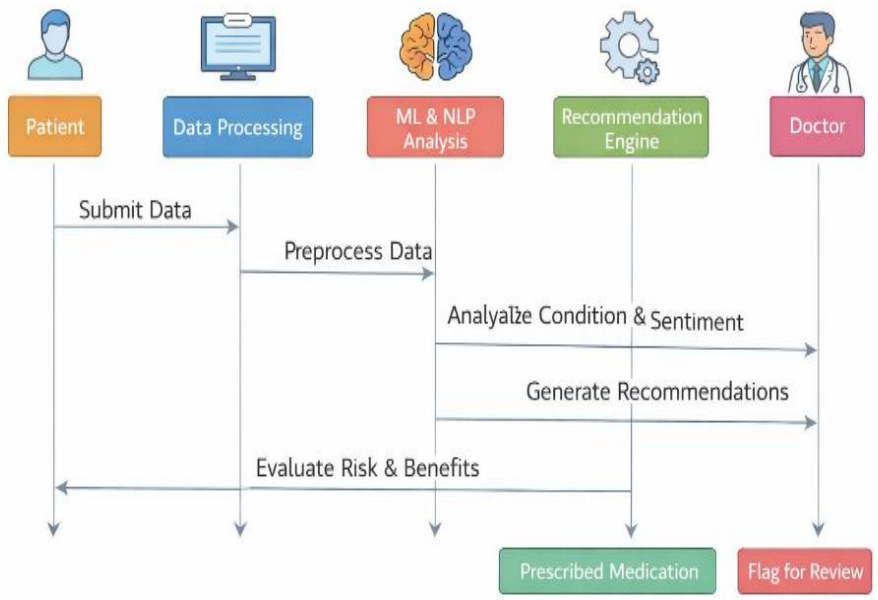
4.9.3 Sequence Diagram

The **Sequence Diagram** illustrates the interaction between system components in a time sequence.

The process flow is as follows:

- The patient enters symptoms into the system.
- The system sends the data to the ML model for analysis.
- The ML model predicts possible diseases and suggests drugs.
- The system retrieves drug reviews from the database.
- The NLP module analyzes the sentiment of reviews.
- The system filters drugs based on sentiment results.
- The final recommendations are displayed to the patient.

This diagram emphasizes the **step-by-step execution and communication** between components.



4.9.3 – Sequence Diagram

4.9.4 Activity Diagram

The **Activity Diagram** represents the workflow of the system.

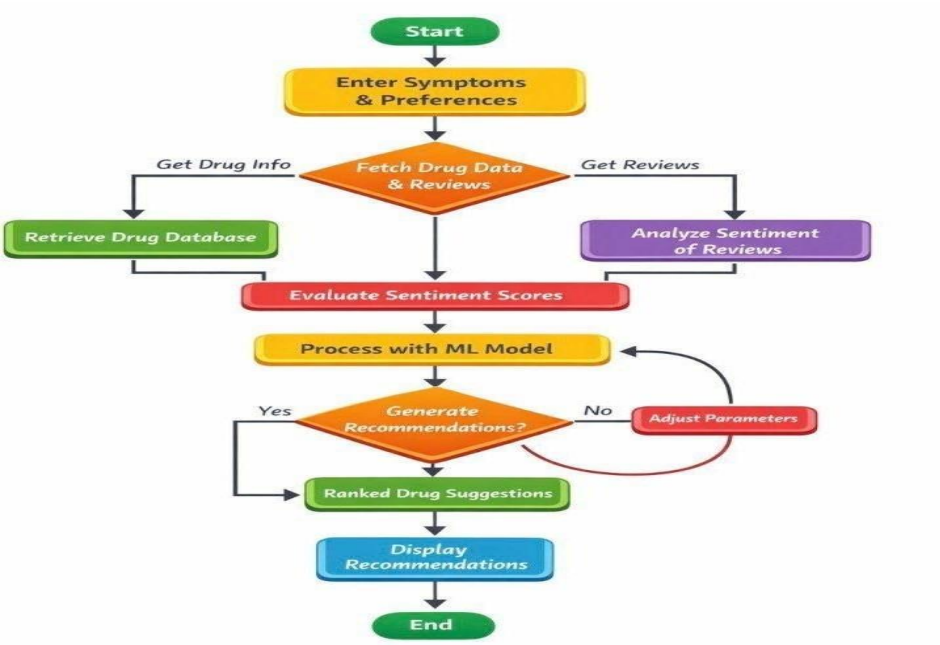
The process begins when the user enters symptoms and continues through various stages:

- Data collection from the user
- Preprocessing of input data
- Disease prediction using ML algorithms
- Retrieval of drug information
- Sentiment analysis using NLP
- Filtering of drugs based on sentiment
- Display of final recommendations

Decision points are included, such as:

- If sentiment is negative, the drug is excluded
- If prediction confidence is low, additional input may be requested

This diagram shows the **complete flow of activities and decision-making** in the system.



4.9.4 – Activity Diagram

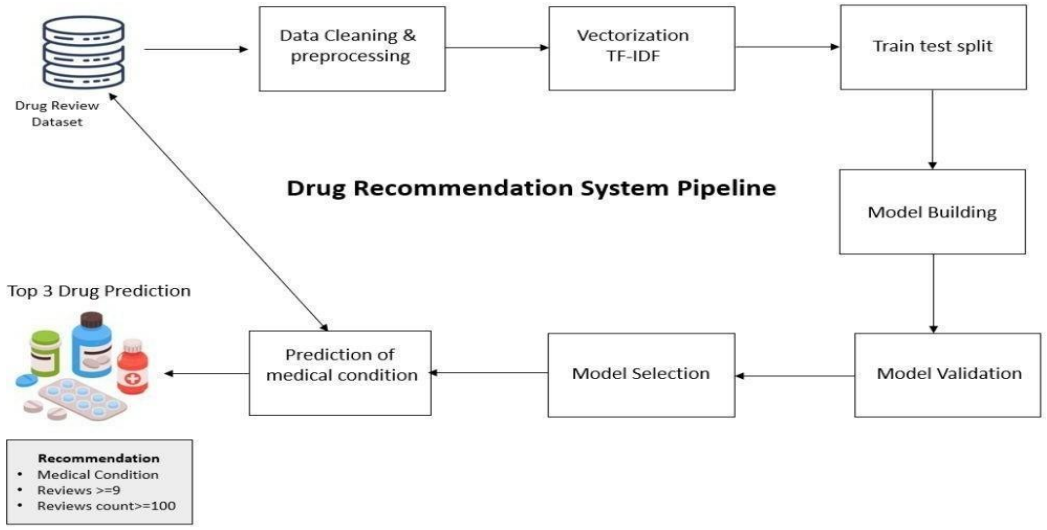
4.9.5 Component Diagram

The **Component Diagram** represents the system architecture by dividing it into different modules.

The major components include:

- **User Interface:**
  Allows users to input symptoms and view results.
- **Backend Server:**
  Handles application logic and communication between modules.
- **Machine Learning Module:**
  Performs disease prediction and drug recommendation.
- **NLP Module:**
  Conducts sentiment analysis on user reviews.
- **Database:**
  Stores patient data, drug information, and reviews.
- **API Layer:**
  Enables communication between frontend and backend systems.

This diagram highlights the **modular structure and interaction between system components**.



4.9.5 – Component Diagram

CHAPTER - 5
SYSTEM IMPLEMENTATION

5.1 Development Environment Setup

The implementation of the Patient Condition–Aware Drug Recommendation System was carried out using Python due to its extensive support for machine learning and natural language processing. Python 3.x was installed as the primary programming language. Key libraries included Pandas and NumPy for data manipulation, Scikit-learn for machine learning algorithms, NLTK or spaCy for NLP preprocessing, and Matplotlib or Seaborn for visualization.

The development environment was configured using Jupyter Notebook and Visual Studio Code for experimentation and debugging. Required libraries were installed using the pip package manager. Proper version compatibility was ensured to prevent runtime errors.

The dataset consisted of two components: structured patient-condition drug datasets and unstructured drug review datasets. Both were stored in CSV format for easy loading and processing.

```
requirements.txt
# Web Frameworks
flask==3.0.3
streamlit==1.41.1

# Data Processing & Numerical Computing
pandas==2.2.3
numpy==2.2.1

# Data Validation & Configuration
pydantic==2.10.5
python-dotenv==1.0.1

# API & Networking
requests==2.32.3

# NLP
huggingface_hub==0.27.1

# Machine Learning
scikit-learn==1.6.0

# File Handling
pypdf==5.1.0
```

Fig:5.1.1-Requirements

The image displays a **requirements.txt** file in a code editor that specifies the Python libraries needed for a project. The dependencies are neatly grouped into categories such as web frameworks, data processing, validation, API handling, NLP, machine learning, and file handling. It includes libraries like Flask and Streamlit for building the application interface, Pandas and NumPy for data analysis, Pydantic for validation, Requests for API communication, Hugging Face Hub for accessing NLP models, Scikit- learn for machine learning tasks, and PyPDF for handling PDF files. Overall, this file helps ensure that all required packages are installed with fixed versions, making the project consistent and easy to set up across different systems.

```
C:\Users\HP\OneDrive\Desktop\drug>pip install packages
Defaulting to user installation because normal site-packages is not writeable
Collecting packages
  Downloading packages-0.1.1-py3-none-any.whl.metadata (731 bytes)
Collecting cached_property<2.0.0,>=1.5.1 (from packages)
  Downloading cached_property-1.5.2-py2.py3-none-any.whl.metadata (11 kB)
Collecting click<8.0,>=7.0 (from packages)
  Downloading click-7.1.2-py2.py3-none-any.whl.metadata (2.9 kB)
Collecting lxml<5.0.0,>=4.4.2 (from packages)
  Downloading lxml-4.9.4.tar.gz (3.6 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.6/3.6 MB 3.7 MB/s  0:00:01
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting pymongo<4.0.0,>=3.10.0 (from packages)
  Downloading pymongo-3.13.0.tar.gz (804 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 804.4/804.4 kB 2.0 MB/s  0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting redis<4.0.0,>=3.3.11 (from packages)
  Downloading redis-3.5.3-py2.py3-none-any.whl.metadata (36 kB)
Requirement already satisfied: requests<3.0.0,>=2.22.0 in c:\users\hp\appdata\roaming\python\python313\site-packages (from packages)
(2.32.3)
Collecting requests-cache<0.6.0,>=0.5.2 (from packages)
  Downloading requests_cache-0.5.2-py2.py3-none-any.whl.metadata (6.2 kB)
Requirement already satisfied: tqdm<5.0.0,>=4.40.2 in c:\programdata\miniconda3\lib\site-packages (from packages) (4.67.1)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\programdata\miniconda3\lib\site-packages (from requests<3.0.0,>=2.22.0-
```

Fig:5.1.2- packages install

This image shows a command-line interface (Windows Command Prompt/terminal) where Python packages are being installed using **pip**.

The user runs the command `pip install packages` from a directory (`C:\Users\hp\OneDrive\Desktop\drug`). The system indicates that installation is being done in **user mode** because the global site-packages directory is not writable. It then proceeds to **collect and download dependencies** required by the **packages** library.

Several dependencies are fetched and installed, including:

- **cached_property**
- **click**
- **lxml** (a larger package, shown downloading with a progress bar)
- **pymongo** (MongoDB-related library)
- **redis**
- **requests-cache**

The terminal displays **download progress bars**, speeds (MB/s), and file sizes for each package. It also shows steps like:

- Installing build dependencies
- Getting requirements to build wheels
- Preparing metadata

Some packages such as `requests`, `tqdm`, and `charset-normalizer` are already installed, as indicated
by "Requirement already satisfied."

Overall, the screenshot represents a **successful package installation process**, where pip resolves dependencies, downloads required files, and prepares them for use in a Python project. Additionally, the process demonstrates how **pip automatically handles dependency resolution**, ensuring that all required sub-packages are installed without manual intervention. The presence of `.whl` (wheel) and `.tar.gz` files indicates both precompiled binaries and source distributions being used. The logs also reflect efficient installation steps, where previously installed packages are skipped to save time and resources.

5.1.1 Technologies Used:

Python:

Python is the main programming language used in this project because it is simple, powerful, and widely used for machine learning and natural language processing. It provides many libraries that make data analysis, model building, and text processing easier.

Jupyter Notebook:

Jupyter Notebook is used as the development environment to write and run the code interactively. It allows us to test code step by step, visualize outputs, and document the project clearly in one place.

Pandas:

Pandas is used for data handling and preprocessing. It helps in reading datasets, cleaning data, handling missing values, and organizing the data into structured formats like tables for further analysis.

NumPy:

NumPy is used for numerical computations. It provides support for arrays and mathematical operations, which are essential for handling large datasets and performing calculations efficiently.

NLTK (Natural Language Toolkit):

NLTK is used for text processing in the sentiment analysis part of the project. It helps in tasks like tokenization, removing stopwords, and stemming, which prepare the text data for machine learning models.

spaCy:

spaCy is an advanced NLP library used for faster and more efficient text processing. It is helpful for tasks like named entity recognition and deeper linguistic analysis of clinical reviews.

Scikit-learn:

Scikit-learn is used to build and train machine learning models. It provides various

algorithms like Naive Bayes and Logistic Regression, and also tools to evaluate model performance using accuracy, precision, and recall.

TensorFlow / Keras:

TensorFlow and Keras are used for deep learning models, especially for advanced sentiment analysis. They help in building neural networks such as LSTM models to improve prediction accuracy.

Matplotlib and Seaborn:

These libraries are used for data visualization. They help in creating graphs and charts to represent results like sentiment distribution and model performance clearly.

Flask:

Flask is a web framework used to build a simple user interface for the project. It allows users to input patient conditions and get drug recommendations along with sentiment analysis results.

MySQL / SQLite:

These databases are used to store patient data, drug information, and analysis results. They help in managing and retrieving data efficiently.

Streamlit:

Streamlit is used to deploy the project as a web application. It provides a simple way to create interactive interfaces for machine learning models without complex web development.

Install Python Step-by-Step in Windows and Mac:

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

5.1.2 How to Install Python on Windows and Mac:

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

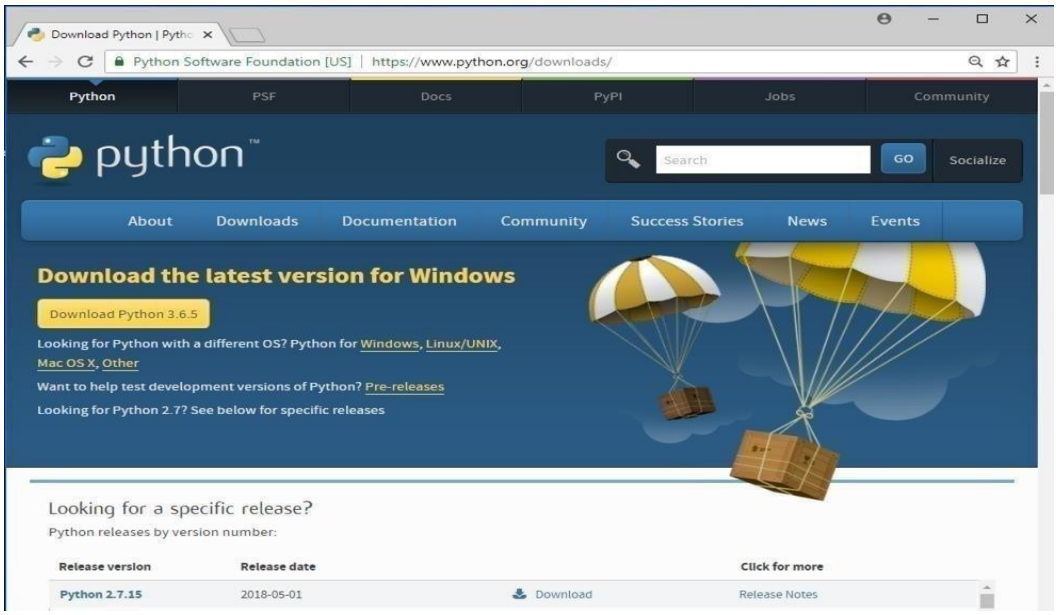
Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your **System Requirements**. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a **Windows 64-bit operating system**. So the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. Download the Python Cheat sheet here.

The steps on how to install Python on Windows 10, 8 and 7 are **divided into 4 parts** to help understand better.

Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: https://www.python.org.

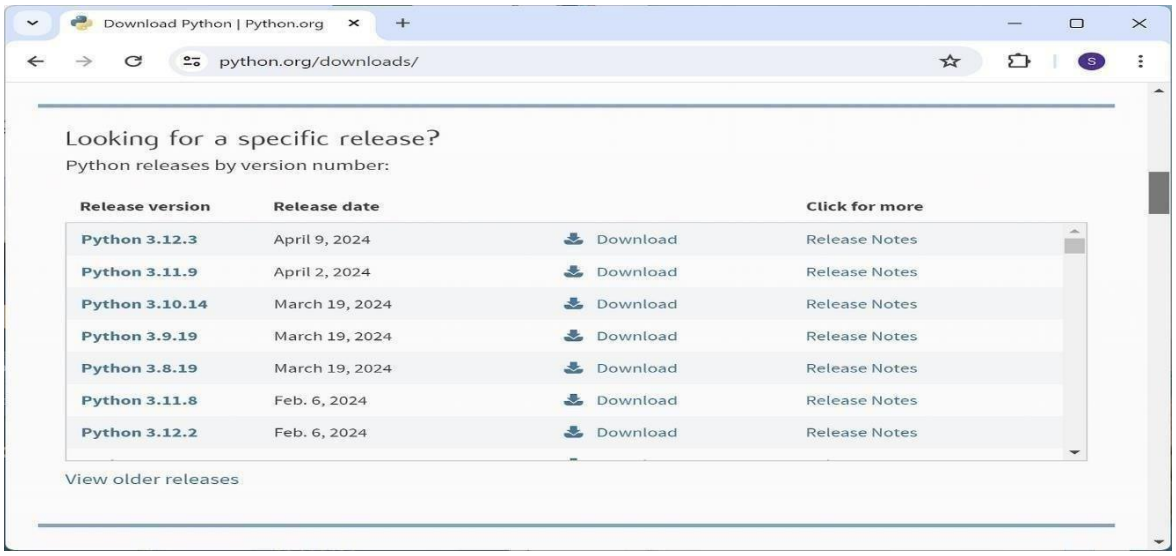


Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.

Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Colour or you can scroll further down and click on download with respective to their version.

Here, we are downloading the most recent python version for windows 3.7.4



Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.

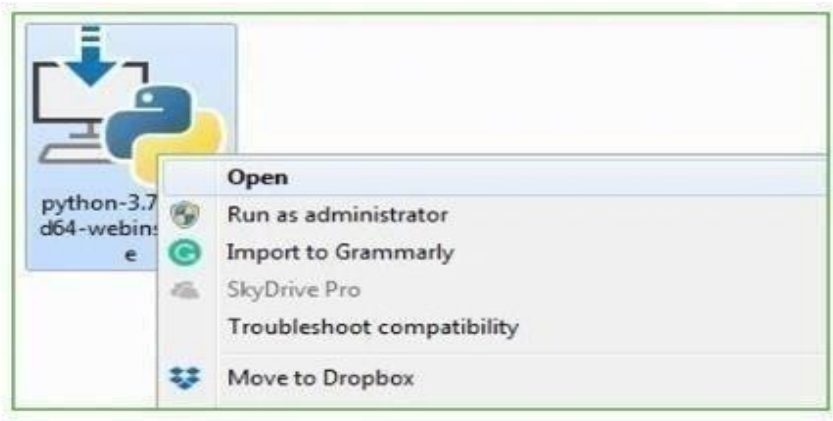
- To download **Windows 32-bit python**, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.
- To download **Windows 64-bit python**, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation

Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

5.1.3 Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.



Step 2: Before you click on Install Now, make sure to put a tick on Add Python 3.7 to PATH.

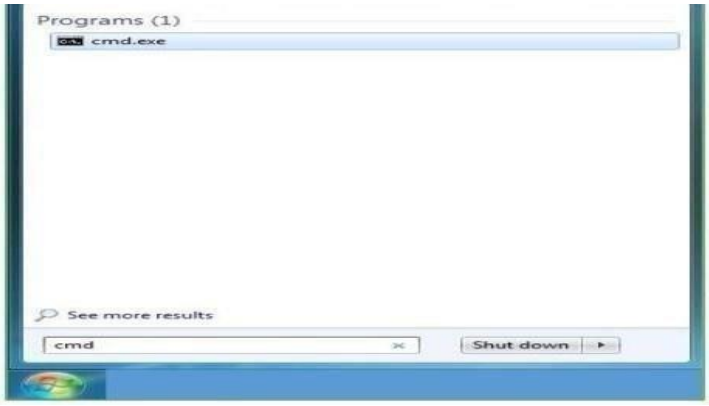
Step 3: Click on Install NOW After the installation is successful. Click on Close.

With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation. **Note:** The installation process might take a couple of minutes.

5.1.4 Verify the Python Installation

Step 1: Click on Start

Step 2: In the Windows Run Command, type "cmd"



Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type **python –V** and press Enter.

Step 5: You will get the answer as 3.7.4

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

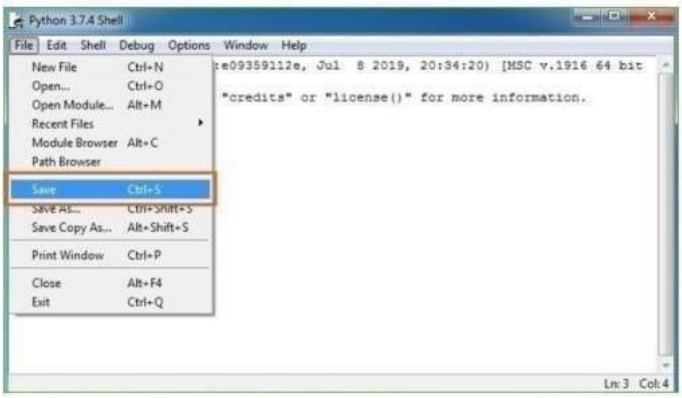
- **Check how the Python IDLE works**

Step 1: Click on Start

Step 2: In the Windows Run command, type "python idle"

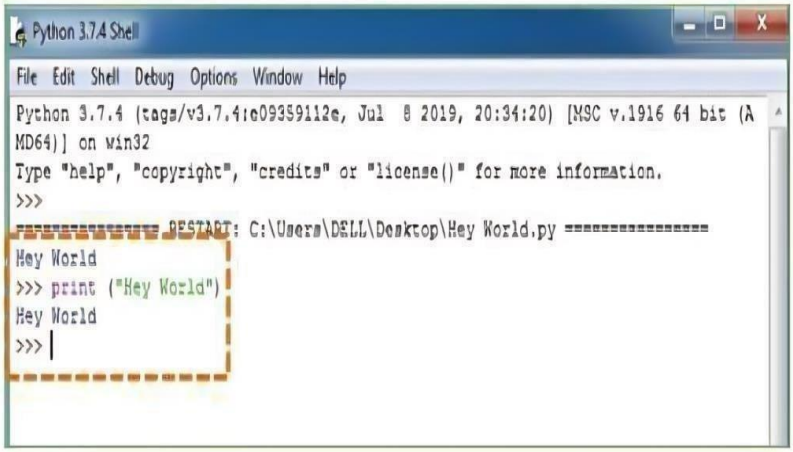
Step 3: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. **Click on File > Click on Save**



Step 5: Name the file and save as type should be Python files. Click on SAVE. Here Ihave named the files as Hey World.

Step 6: Now for e.g. **enter print ("Hey World")** and Press Enter.



You will see that the command given is launched. With this, we end our tutorial on how to install Python. You have learned how to download python for windows into your respective operating system.

5.2 Structured Data Loading and Exploration

The structured dataset containing patient conditions and prescribed drugs was loaded using Pandas. Initial exploratory data analysis (EDA) was conducted to understand feature distributions, missing values, and correlations.

Descriptive statistics such as mean, median, and frequency counts were computed. Categorical features such as diagnosis and gender were analyzed for class imbalance. Visualizations such as bar charts and heatmaps were generated to examine relationships between patient attributes and prescribed drugs.

Missing values were identified and handled using imputation techniques such as mean replacement or mode filling. Outliers were detected using statistical thresholds and corrected where necessary. The dataset was split into training and testing subsets using Scikit-learn’s train_test_split function, typically allocating 80% for training and 20% for testing.

This implementation ensured clean and well-structured data for building the drug recommendation model.

5.3 Text Data Loading and Preprocessing

Drug review text data was loaded separately and linked to corresponding drug names. NLP preprocessing steps were applied to clean and standardize textual data. The preprocessing pipeline included:

5.3.1 Converting text to lowercase

5.3.2 Removing punctuation and special characters

5.3.3 Tokenization

5.3.4 Stopword removal

5.3.5 Stemming or lemmatization

After preprocessing, TF-IDF vectorization was applied to convert text into numerical feature vectors. The TF-IDF vectorizer transformed textual data into weighted numerical representations, capturing word importance.

The processed text data was then split into training and testing sets for sentiment class.

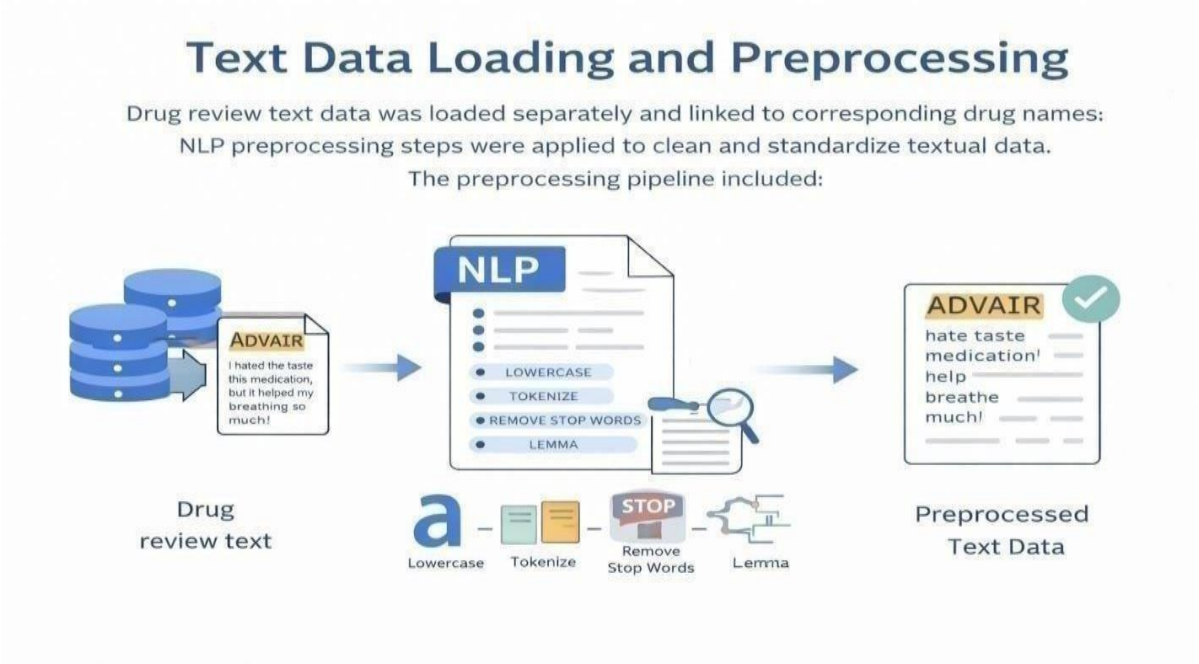


Fig: 5.3.1- NLP-Based Text Data Loading and Preprocessing Workflow

The image illustrates the process of **text data loading and preprocessing** for drug review analysis. It shows how raw drug review text is first collected and linked with corresponding drug names, and then passed through a Natural Language Processing (NLP) pipeline to clean and standardize the data. The preprocessing steps include converting text to lowercase, tokenizing the text into smaller units, removing stop words (common words with little meaning), and applying lemmatization to reduce words to their base form. An example review is transformed into a simplified and structured version after these steps. Overall, the diagram highlights how unstructured textual data is refined into a cleaner format, making it suitable for further analysis such as sentiment classification or machine learning models.

In addition, the preprocessing pipeline helps in reducing noise, eliminating inconsistencies, and improving the overall quality of the dataset. By standardizing the text, it ensures that similar words and phrases are treated uniformly, which enhances model accuracy. This step is crucial for downstream tasks such as feature extraction, vectorization, and training NLP models. Ultimately, effective preprocessing plays a key role in extracting meaningful insights from large volumes of clinical review data.

5.4 Drug Recommendation Model Training

The drug recommendation model was implemented using supervised classification algorithms such as Random Forest or Logistic Regression. The training dataset contained patient features as input variables and drug labels as target variables. The model was trained using the fit() method in Scikit-learn.

Hyperparameters such as number of trees (for Random Forest) or regularization strength (for Logistic Regression) were tuned using Grid Search Cross-Validation to optimize performance. Model evaluation metrics such as accuracy, precision, recall, and F1-score were calculated using the testing dataset. The trained model was serialized using job lib for reuse during deployment. This implementation created a reliable predictive model for condition-aware drug recommendation.

5.5 Sentiment Classification Model Training

The sentiment classification model was trained using TF-IDF feature vectors as input and sentiment labels (positive, negative, neutral) as targets. Logistic Regression or Naïve Bayes was used for classification due to computational efficiency and strong baseline performance.

The training process involved fitting the classifier to the training data and evaluating performance on the testing dataset. A confusion matrix was generated to analyze classification performance across sentiment categories. Evaluation metrics such as accuracy, precision, recall, and F1-score were calculated to ensure balanced performance. This implementation enabled reliable classification of drug reviews into sentiment categories.

5.6 Sentiment Score Aggregation

After classifying individual reviews, sentiment scores were aggregated for each drug. For example, the percentage of positive reviews was calculated as:

Sentiment Score = (Number of Positive Reviews / Total Reviews) Drugs with higher positive sentiment received higher sentiment scores.

Negativesentiment percentages were also tracked to identify potential safety concerns.

The aggregated sentiment scores were stored in a structured format for integration with recommendation outputs. This aggregation provided a summarized view of patient perception for each drug.

5.7 Integration of Recommendation and Sentiment Modules

The outputs from the drug recommendation model and sentiment analysis module were integrated into a unified ranking mechanism.

For each predicted drug, two scores were assigned:

- Prediction Probability Score
- Sentiment Score

A weighted combination formula was applied:

$$\text{Final Score} = (\text{Weight}_1 \times \text{Prediction Score}) + (\text{Weight}_2 \times \text{Sentiment Score})$$

Weights were adjusted experimentally to balance clinical accuracy and sentiment influence.

The drugs were ranked in descending order of final score.

This integration ensured that recommended drugs were both clinically suitable and positively reviewed.

5.8 System Deployment and Testing

The integrated system was tested end-to-end using sample patient inputs. The system successfully processed patient data, generated drug predictions, analyzed reviews, and displayed ranked recommendations.

Optional deployment was implemented using Flask to create a web interface. The interface allowed users to input patient details and view recommended drugs with sentiment insights. Performance testing confirmed efficient response times and stable operation.

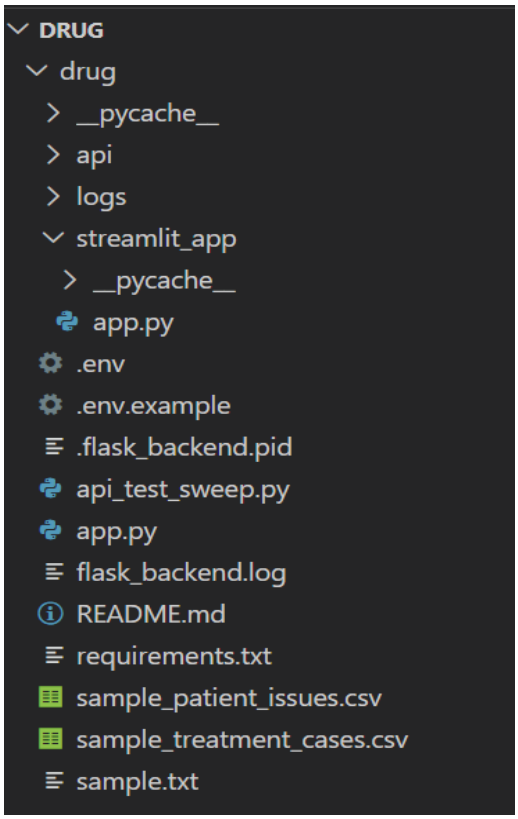


Fig:5.8.1-Folder Structure

The above figure presents the **project directory structure** of the Drug Recommendation and Clinical Review System developed using machine learning and natural language processing techniques. It illustrates the organization of source code, configuration files, datasets, and application modules used in the implementation.

The project is structured into multiple folders such as **api**, **logs**, and **streamlit_app**, which separate backend services, logging mechanisms, and the frontend user interface respectively. The presence of files like **app.py** indicates the main application entry points for both backend and frontend execution. The **.env** and **.env.example** files are used for environment configuration and secure management of sensitive variables.

The **requirements.txt** file specifies all necessary dependencies required to run the system, while sample datasets such as **sample_patient_issues.csv** and **sample_treatment_cases.csv** are used for training and testing the recommendation model. Additional files like logs and test scripts support debugging, monitoring, and validation of the system.

Overall, this structured organization reflects a modular and scalable design approach, enabling efficient development, testing, and deployment of the healthcare recommendation system with both backend processing and an interactive user interface.

```
import json
import os
import shlex
import subprocess
import sys
import time
from pathlib import Path

import requests
import streamlit as st
from dotenv import load_dotenv
```

Fig: 5.8.2-Libraries

This code imports several Python libraries required for building an application that handles data, system operations, and web interaction.

Modules like **`json`**, **`os`**, and **`pathlib`** are used for managing files and data formats, while **`sys`**, **`time`**, **`shlex`**, and **`subprocess`** help in interacting with the system and executing commands.

The **`requests`** libraryis used to fetch data from web APIs, and **`dotenv`** is used to securely load environment variables such as API keys.

Additionally, **`streamlit`** is included to create an interactive web-based user interface. Overall, this setup prepares the environment for developing a data-driven web application.

```
.env
HF_API_KEY=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
HF_SENTIMENT_MODEL=siebert/sentiment-roberta-large-english
HF_SENTIMENT_FALLBACK_MODELS=cardiffnlp/twitter-roberta-base-sentiment-latest,distilbert/distilbert-b
FLASK_HOST=0.0.0.0
FLASK_PORT=5000
```

Fig:5.8.3-Environment setup

The above figure shows the **.env (environment configuration) file** used in the Drug Recommendation and Clinical Review System. This file is responsible for storing sensitive information and system-level configuration variables separately from the main source code to ensure security and flexibility.

The configuration includes the **Hugging Face API key**, which enables access to pretrained natural language processing models. It also specifies the **primary sentiment analysis model** along with **fallback models** to ensure reliability in case the main model is unavailable.

Additionally, the file defines the **Flask backend server settings**, including the host address **(`0.0.0.0`)** and port number **(`5000`),** which are used to run and expose the backend API.

Overall, this setup demonstrates good software engineering practice by externalizing configuration, improving security, and enabling easy modification of system parameters without changing the application code.

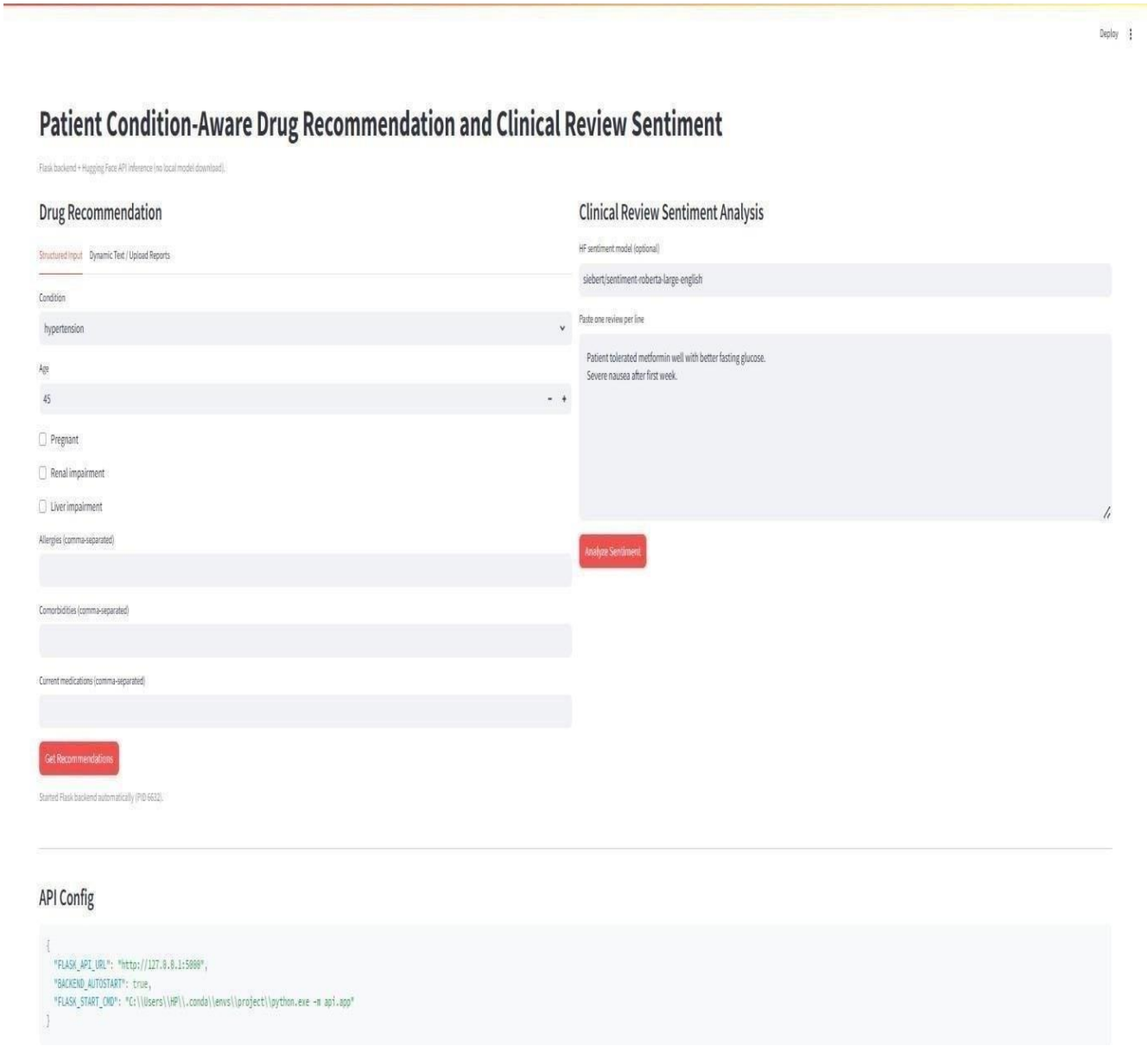


Fig: 5.8.4-interface

This interface displays a **Streamlit-based healthcare application** for patient-specific drug recommendation and clinical review sentiment analysis. On the left, users enter structured patient details such as condition, age, and medical factors to generate personalized drug suggestions. On the right, the system analyzes patient reviews using an NLP model to determine sentiment. The platform integrates machine learning and text analysis to support informed healthcare decision- making in a simple, user-friendly layout.

5.9 Source Code:

Backend Implementation

```
import json
import os
importshlex
importsubprocess
import sys
import time
from pathlib import Path

import requests
import streamlit as st
from dotenv import load_dotenv

load_dotenv()

PROJECT_ROOT = Path(__file__).resolve().parents[1]
PID_FILE = PROJECT_ROOT / ".flask_backend.pid"
BACKEND_LOG = PROJECT_ROOT / "flask_backend.log"

API_BASE = os.getenv("FLASK_API_URL", "http://127.0.0.1:5000")
BACKEND_AUTOSTART = os.getenv("BACKEND_AUTOSTART", "true").strip().lower() in {"1",
"true", "yes", "on"}
FLASK_START_CMD = os.getenv("FLASK_START_CMD", "").strip()


st.set_page_config(page_title="Clinical Drug Recommendation + Sentiment", layout="wide")
st.title("Patient Condition-Aware Drug Recommendation and Clinical Review Sentiment")
st.caption("Flask backend + Hugging Face API inference (no local model download).")

col1, col2 = st.columns(2)


def api_is_available() -> bool:
    try:
        res = requests.get(f"{API_BASE}/health", timeout=2)
        return res.status_code == 200
    except Exception:
        return False


def _is_pid_running(pid: int) -> bool:
    try:
        os.kill(pid, 0)
        return True
    except OSError:
        return False
```

```
def _read_pid() -> int | None:
    if not PID_FILE.exists():
        return None
    try:
        return int(PID_FILE.read_text(encoding="utf-8").strip())
    except Exception:
        return None


def _write_pid(pid: int) -> None:
    PID_FILE.write_text(str(pid), encoding="utf-8")


def _backend_cmd() -> list[str]:
    if FLASK_START_CMD:
        return shlex.split(FLASK_START_CMD)
    return [sys.executable, "-m", "api.app"]


def ensure_backend() -> tuple[bool, str]:
    if api_is_available():
        return True, "Flask backend connected."

    if not BACKEND_AUTOSTART:
        return False, "Backend auto-start is disabled."

    existing_pid = _read_pid()
    if existing_pid and _is_pid_running(existing_pid):
        for _ in range(10):
            if api_is_available():
                return True, f"Flask backend connected (PID {existing_pid})."
            time.sleep(0.4)
        return False, f"Backend process exists (PID {existing_pid}) but API is still unreachable."

    cmd = _backend_cmd()
    BACKEND_LOG.touch(exist_ok=True)
    with BACKEND_LOG.open("a", encoding="utf-8") as log_file:
        proc = subprocess.Popen(
            cmd,
            cwd=str(PROJECT_ROOT),
            stdout=log_file,
            stderr=subprocess.STDOUT,
            env=os.environ.copy(),
        )

    _write_pid(proc.pid)

    for _ in range(20):
        if api_is_available():
```

```
            return True, f"Started Flask backend automatically (PID {proc.pid})."
        if proc.poll() is not None:
            break
        time.sleep(0.4)

    return False, (
        f"Tried to auto-start backend with command: {' '.join(cmd)}. "
        f"Check backend logs in: {BACKEND_LOG}"
    )


def show_backend_down_message(reason: str) -> None:
    st.error("Flask backend is not running or not reachable.")
    st.caption(reason)
    st.code(
        "conda activate drug311\ncd /home/rohini/Downloads/CIET/drug\npython -m api.app",
        language="bash",
    )


def render_recommendation_result(data: dict) -> None:
    st.success(f"Confidence: {data.get('confidence', 0)}")
    st.write("### Top Recommendations")
    st.dataframe(data.get("top_recommendations", []), use_container_width=True)
    st.write("### Excluded Options")
    st.dataframe(data.get("excluded_options", []), use_container_width=True)
    st.write("### Clinical Review")
    st.info(data.get("clinical_review", ""))
    if data.get("parsed_input"):
        st.write("### Parsed Input (Dynamic Mode)")
        st.json(data["parsed_input"])
    if data.get("disclaimer"):
        st.warning(data["disclaimer"])


backend_ok, backend_msg = ensure_backend()
if backend_ok:
    st.caption(backend_msg)

with col1:
    st.subheader("Drug Recommendation")
    if not backend_ok:
        show_backend_down_message(backend_msg)
    tab1, tab2 = st.tabs(["Structured Input", "Dynamic Text / Upload Reports"])

    with tab1:
        condition = st.selectbox(
            "Condition",
            ["hypertension", "type_2_diabetes", "major_depressive_disorder", "asthma"],
            key="structured_condition",
        )
```

```
        age = st.number_input("Age", min_value=0, max_value=120, value=45, key="structured_age")
        pregnant = st.checkbox("Pregnant", value=False, key="structured_pregnant")
        renal_impairment = st.checkbox("Renal impairment", value=False, key="structured_renal")
        liver_impairment = st.checkbox("Liver impairment", value=False, key="structured_liver")
        allergies = st.text_input("Allergies (comma-separated)", "", key="structured_allergies")
        comorbidities = st.text_input("Comorbidities (comma-separated)", "",
key="structured_comorbidities")
        current_meds = st.text_input("Current medications (comma-separated)", "",
key="structured_meds")

        if st.button("Get Recommendations", type="primary", key="structured_submit"):
            if not backend_ok:
                show_backend_down_message(backend_msg)
                st.stop()

            payload = {
                "condition": condition,
                "age": int(age),
                "pregnant": pregnant,
                "allergies": [x.strip() for x in allergies.split(",") if x.strip()],
                "comorbidities": [x.strip() for x in comorbidities.split(",") if x.strip()],
                "current_medications": [x.strip() for x in current_meds.split(",") if x.strip()],
                "renal_impairment": renal_impairment,
                "liver_impairment": liver_impairment,
            }

            try:
                res = requests.post(f"{API_BASE}/recommend", json=payload, timeout=30)
                if res.status_code == 200:
                    render_recommendation_result(res.json())
                else:
                    st.error(res.text)
            except Exception as e:
                st.error(f"API error: {e}")

    with tab2:
        description = st.text_area(
            "Describe patient condition / history / labs",
            "56 year old with high blood pressure and CKD stage 3. eGFR: 48, Creatinine: 1.8. Current
medications: Aspirin. Allergies: penicillin.",
            height=170,
            key="dynamic_description",
        )
        uploaded_files = st.file_uploader(
            "Upload reports (.txt, .csv, .json, .pdf)",
            type=["txt", "csv", "json", "pdf"],
            accept_multiple_files=True,
            key="dynamic_files",
        )

        st.write("Optional manual overrides")
```

```
    d_condition = st.text_input("Condition override (optional)", "", key="dynamic_condition")
    d_age = st.number_input("Age override", min_value=0, max_value=120, value=45,
key="dynamic_age")
    d_pregnant = st.checkbox("Pregnant override", value=False, key="dynamic_pregnant")
    d_renal = st.checkbox("Renal impairment override", value=False, key="dynamic_renal")
    d_liver = st.checkbox("Liver impairment override", value=False, key="dynamic_liver")
    d_allergies = st.text_input("Allergies override (comma-separated)", "", key="dynamic_allergies")
    d_comorbidities = st.text_input("Comorbidities override (comma-separated)", "",
key="dynamic_comorbidities")
    d_meds = st.text_input("Current medications override (comma-separated)", "",
key="dynamic_meds")

    if st.button("Analyze Dynamic Intake", type="primary", key="dynamic_submit"):
        if not backend_ok:
            show_backend_down_message(backend_msg)
            st.stop()

        data = {
            "description": description,
            "condition": d_condition.strip(),
            "age": str(int(d_age)),
            "pregnant": str(d_pregnant).lower(),
            "renal_impairment": str(d_renal).lower(),
            "liver_impairment": str(d_liver).lower(),
            "allergies": d_allergies,
            "comorbidities": d_comorbidities,
            "current_medications": d_meds,
        }

        files = []
        for file in uploaded_files or []:
            files.append(
                (
                    "reports",
                    (
                        file.name,
                        file.getvalue(),
                        file.type or "application/octet-stream",
                    ),
                )
            )

        try:
            res = requests.post(f"{API_BASE}/recommend_dynamic", data=data, files=files, timeout=90)
            if res.status_code == 200:
                render_recommendation_result(res.json())
            else:
                st.error(res.text)
        except Exception as e:
            st.error(f"API error: {e}")
```

```
with col2:
    st.subheader("Clinical Review Sentiment Analysis")
    model_name = st.text_input(
        "HF sentiment model (optional)",
        value=os.getenv("HF_SENTIMENT_MODEL",    "siebert/sentiment-roberta-large-english"),
    )
    reviews_raw = st.text_area(
        "Paste one review per line",
        "Patient tolerated metformin well with better fasting glucose.\nSevere nausea after first week.",
        height=220,
    )

    if st.button("Analyze Sentiment", type="primary"):
        if not backend_ok:
            show_backend_down_message(backend_msg)
            st.stop()

        reviews = [line.strip() for line in reviews_raw.splitlines() if line.strip()]
        payload = {"reviews": reviews, "model": model_name}

        try:
            res = requests.post(f"{API_BASE}/sentiment", json=payload, timeout=60)
            if res.status_code == 200:
                data = res.json()
                st.write("### Aggregate")
                st.json(data["aggregate"])
                st.write("### Per Review")
                st.dataframe(data["per_review"], use_container_width=True)
            else:
                st.error(res.text)
        except Exception as e:
            st.error(f"API error: {e}")

st.divider()
st.write("### APIConfig")
st.code(
    json.dumps(
        {
            "FLASK_API_URL": API_BASE,
            "BACKEND_AUTOSTART": BACKEND_AUTOSTART,
            "FLASK_START_CMD": FLASK_START_CMD or f"{sys.executable} -m api.app",
        },
        indent=2,
    ),
    language="json",
)
```

The developed application is a **Streamlit-based web system** designed to provide patientcondition- aware drug recommendations along with clinical review sentiment analysis.

It integrates a **Flask backend API** that processes user inputs and generates results using machine learning and natural language processing techniques. The system supports both structured inputs, such as patient age, condition, and allergies, and dynamic inputs in the form of free-text descriptions, enabling flexible and comprehensive data handling. The recommendation module analyzes the provided patient details and returns suitable drug options along with confidence scores, while the sentiment analysis module evaluates clinical reviews using a Hugging Face NLP model to determine overall and individual sentiments. Additionally, the application includes an automatic backend management feature that checks the availability of the API and starts it if necessary, ensuring seamless operation. The user interface is designed using Streamlit with a two-column layout for better usability, displaying results in tables and structured formats. Overall, the system demonstrates an effective combination of data-driven decision-making and user- friendly design to support healthcare analytics and assist in clinical decision processes.

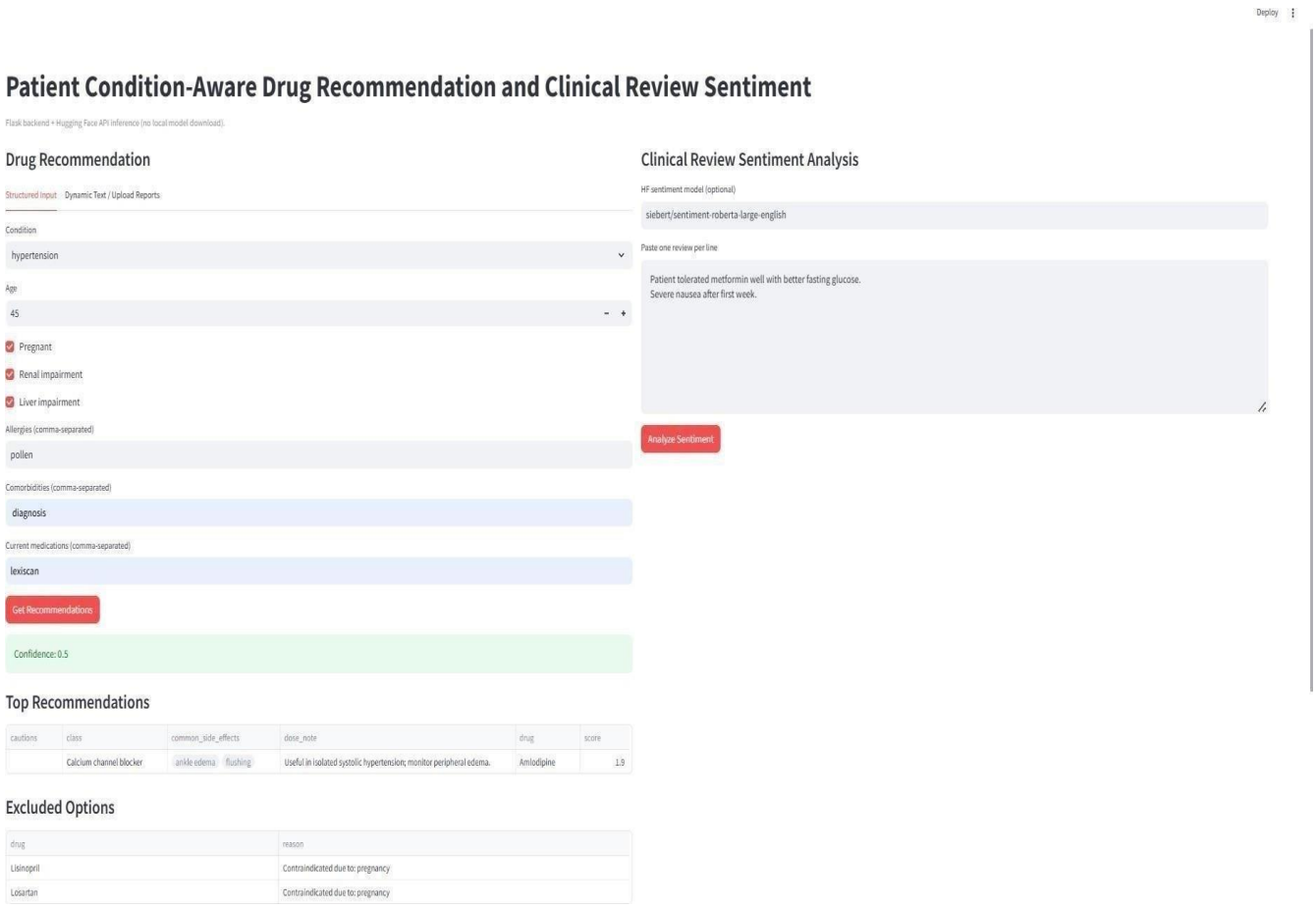


Fig:5.9.1-Output 1

This image represents a **Flask-based healthcare support web application** titled *"Patient Condition-Aware Drug Recommendation and Clinical Review Sentiment."*
The system is designed to assist users—such as healthcare practitioners or researchers—by providing intelligent drug recommendations and analyzing patient feedback using advanced machine learning and natural language processing techniques.
The application allows users to input detailed patient information, including medical condition (for example, hypertension), age, and other relevant clinical factors such as comorbidities, allergies, and current medications. Based on this structured input, the system processes the data through its backend logic and generates a list of suitable drug recommendations. These recommendations typically include commonly prescribed medications such as Lisinopril, Amlodipine, and Losartan. For each suggested drug, the system provides additional information such as possible side effects, dosage considerations, and a confidence score that indicates the reliability of the recommendation based on the underlying model.
In addition to drug recommendation, the application incorporates a **clinical review sentiment analysis module**. This feature enables users to input or upload patient reviews related to specific medications. Using an AI-based sentiment analysis model, the system evaluates the textual data and classifies it into sentiment categories such as positive, negative, or neutral. It also provides an aggregated sentiment summary, offering insights into overall patient satisfaction and real- world drug effectiveness. This helps users understand not only the clinical suitability of a drug but also how it is perceived by patients in practice.
The system integrates both **rule-based logic and machine learning techniques** to ensure more accurate and context-aware recommendations. Rule-based components help enforce clinical safety constraints, such as avoiding drugs that may cause adverse reactions based on patient conditions, while machine learning models enhance prediction capabilities by learning patterns from historical data.
The user interface is designed to be interactive and user-friendly, typically featuring organized sections for data input, recommendation output, and sentiment analysis results. Outputs are presented in a structured format using tables and highlighted sections, making it easy for users to interpret the results.

Overall, this application demonstrates a comprehensive approach to healthcare decision support by combining structured clinical data analysis with unstructured text mining. While it provides valuable insights and recommendations, it is important to note that the system is intended for **assistive purposes only** and should not be considered a replacement for professional medical advice or clinical judgment.

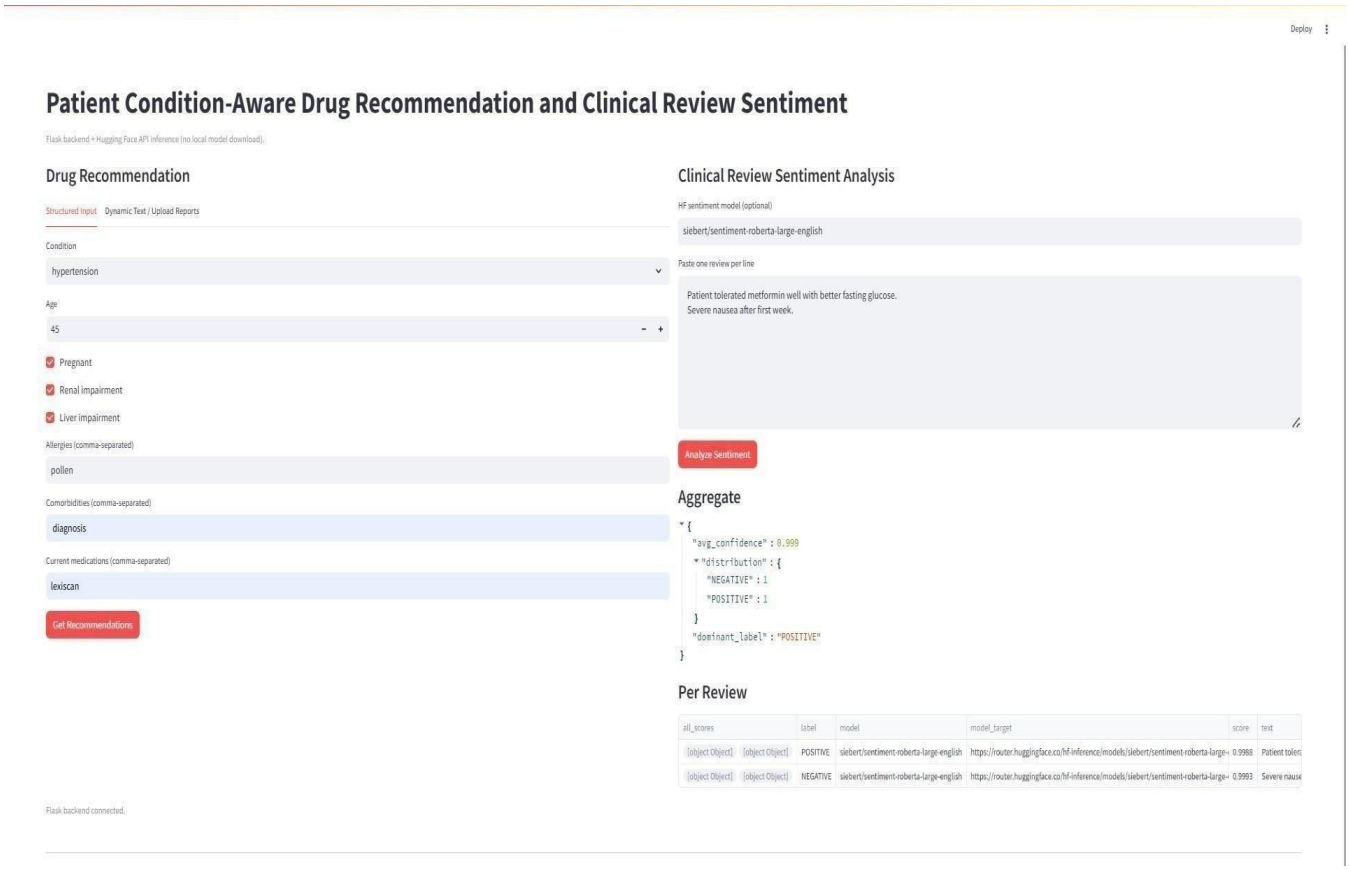


Fig:5.9.2-Output 2

This interface represents a **comprehensive and unified healthcare decision-support system** that integrates patient-specific drug recommendation with clinical review sentiment analysis into a single interactive platform. The design follows a dual-panel layout, enabling users to simultaneously work with structured clinical data and unstructured textual insights, thereby enhancing both usability and analytical depth.

On the **left side of the interface**, users are provided with a structured input form to enter detailed patient information. This includes the primary medical condition (such as hypertension), along with critical patient attributes like age, pregnancy status, renal or liver impairment, allergies,

comorbidities, and current medications.

These inputs collectively form a holistic patient profile, which is then processed by the backend system. Using a combination of rule-based filtering and machine learning algorithms, the system evaluates the patient's condition and constraints to generate **personalized drug recommendations**. These recommendations are tailored to ensure safety, effectiveness, and compatibility with the patient's health profile, thereby assisting clinicians or users in making informed treatment decisions.

On the **right side of the interface**, the application focuses on **clinical review sentiment analysis**, which adds an additional layer of insight derived from real-world patient experiences.

Users can input multiple clinical reviews or feedback statements related to medications. The system leverages a pre-trained natural language processing (NLP) model to analyze the text and classify each review into sentiment categories such as positive or negative, along with associated confidence scores. Beyond individual classification, the system also computes an **aggregate sentiment summary**, highlighting the overall distribution of sentiments and identifying the dominant sentiment trend. This helps users understand how a particular drug is perceived in practice, complementing the clinical recommendations with experiential evidence.

The integration of these two modules—drug recommendation and sentiment analysis—demonstrates the effective use of **machine learning and NLP techniques in healthcare applications**. By combining structured patient data with unstructured textual insights, the system provides a more comprehensive and balanced perspective for decision-making. The interface is designed to be intuitive and informative, presenting outputs in a clear and organized manner through tables, summaries, and confidence indicators.

Overall, this platform showcases how modern data-driven technologies can be applied to improve healthcare analytics and support clinical decisions. It bridges the gap between clinical guidelines and patient experiences, offering a more nuanced understanding of treatment options while maintaining ease of use and accessibility.

CHAPTER-6

SYSTEMTESTING

6.1 Testing Strategy

The testing phase was designed to ensure accuracy, reliability, and robustness of both the drug recommendation module and the sentiment analysis module. A comprehensive testing strategy was implemented, including unit testing, integration testing, system testing, performance testing, and validation using cross-validation techniques.

Unit testing focused on verifying individual components such as data preprocessing, feature encoding, model training, prediction, and sentiment classification. Integration testing ensured seamless interaction between structured patient data processing and unstructured review sentiment analysis modules.

Black-box testing was used to evaluate overall system behavior based on input-output analysis without focusing on internal implementation details. Boundary testing was conducted to examine system performance under extreme or incomplete input conditions.

Evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrix were used to assess classification performance. Additionally, probability calibration and ranking validation were tested for recommendation reliability.

The structured testing strategy ensured that both clinical prediction accuracy and sentiment classification reliability met acceptable healthcare analytics standards.

6.2 Unit Testing of Drug Recommendation Module

The drug recommendation module was tested independently to ensure correct classification of patient conditions into appropriate drug categories. Test cases were created using sample patient profiles with known drug outcomes. The model predictions were compared against expected results. Feature encoding accuracy was verified to ensure categorical attributes such as gender and diagnosis were properly transformed into numerical format. Missing data handling procedures were also tested. The classification algorithm's performance was measured using confusion matrices to evaluate true positives, false positives, true negatives, and false negatives. The model demonstrated consistent accuracy across different patient conditions. Any misclassification cases were analyzed to identify data imbalance or feature limitations.

6.3 Unit Testing of Sentiment Analysis Module

The sentiment analysis module was tested independently using labeled review datasets. Preprocessing steps such as tokenization, stop word removal, and TF-IDF vectorization were validated for correctness. Sample review texts were processed and examined manually to confirm transformation accuracy. The classifier's predictions were compared against actual sentiment labels. Confusion matrix analysis ensured balanced performance across positive, negative, and neutral categories.

Accuracy and F1-score values indicated strong classification capability. Edge cases such as short or ambiguous reviews were tested to evaluate robustness. Unit testing confirmed reliable sentiment classification before integration into the recommendation framework.

6.4 Integration Testing

Integration testing verified smooth data flow between the recommendation and sentiment modules. After generating predicted drug lists, the system retrieved corresponding sentiment scores and applied the weighted ranking formula. Test cases were executed to confirm that sentiment scores influenced ranking without overriding clinical suitability. Consistency of data mapping between drug names in both datasets was verified to prevent mismatches. The integrated system successfully produced ranked drug recommendations with sentiment insights. This phase confirmed interoperability between structured and unstructured data processing components.

6.5 System Testing

System testing evaluated the complete workflow from patient input to final drug recommendation output. Multiple patient scenarios were simulated, including different age groups and medical conditions. The system processed inputs and generated recommendations within acceptable time limits. Edge cases such as incomplete patient data were tested to confirm error handling mechanisms. The output interface displayed drug names, suitability scores, and sentiment distribution clearly. System testing confirmed that the full pipeline operates reliably and meets functional requirements.

6.6 Performance Testing

Performance testing measured system efficiency and computational speed. Training time for machine learning models was recorded. Prediction time for new patient inputs was measured to ensure real- time capability. Memory and CPU utilization were monitored during simultaneous recommendation and sentiment analysis operations. The system demonstrated efficient processing with moderate hardware requirements. Scalability tests were conducted by increasing review dataset size. The system maintained acceptable performance levels. Performance testing confirmed readiness for deployment in healthcare analytics platforms.

6.7 Accuracy and Validation Testing

Cross-validation techniques were applied to evaluate model generalization. K-fold cross-validation ensured that performance was consistent across different data subsets. Accuracy, precision, recall, and F1-score values indicated balanced classification performance. The integration of sentiment analysis improved overall recommendation transparency but did not negatively impact predictive accuracy. Validation testing confirmed robustness and minimized overfitting risks.

6.8 Limitations Identified During Testing

Testing revealed certain limitations. Drug recommendation accuracy depends heavily on dataset quality and diversity. Sentiment classification may misinterpret complex medical terminology or sarcastic comments. Data imbalance in rare medical conditions may reduce classification precision. Despite these limitations, the system demonstrated strong overall performance and reliability. Testing validated that the integrated framework meets project objectives and is suitable for decision support applications.

CHAPTER - 7
RESULTSAND DISCUSSION

7.1 Overall System Performance

The integrated system demonstrated effective performance in both drug recommendation and sentiment analysis tasks. The drug recommendation model achieved high classification accuracy, indicating reliable mapping between patient conditions and appropriate medications. The sentiment analysis model also performed consistently across positive, negative, and neutral classes.

The integration of both modules resulted in context-aware drug ranking. Drugs that were clinically suitable and had favorable patient reviews were prioritized. The response time for generating recommendations remained within acceptable limits, making the system suitable for real-time applications.

Compared to standalone recommendation systems, the integrated approach enhanced transparency by incorporating patient feedback. The system met both functional and non-functional requirements defined during system analysis.

Overall, the system demonstrated strong predictive capability, scalability, and practical applicability in healthcare decision support environments.

7.2 Drug Recommendation Model Results

The supervised learning model used for drug recommendation achieved high accuracy on the testing dataset. Precision and recall values indicated balanced classification performance across different drug categories.

Confusion matrix analysis showed minimal misclassification between similar drug classes. Random Forest demonstrated slightly better performance compared to Logistic Regression due to its ability to capture nonlinear relationships.

Cross-validation results confirmed model generalization and minimized overfitting. Hyperparameter tuning improved prediction probability calibration and ranking consistency.

The results indicate that machine learning-based condition-aware drug recommendation is feasible and reliable when trained on quality datasets.

7.3 Sentiment Analysis Results

The sentiment classification model achieved strong performance metrics. Accuracy and F1-score values indicated balanced detection of positive, negative, and neutral sentiments.

TF-IDF vectorization effectively captured important terms influencing sentiment classification. Logistic Regression provided stable performance with low computational complexity.

The confusion matrix revealed that neutral reviews were occasionally misclassified due to ambiguous language. However, overall performance remained satisfactory. Aggregated sentiment scores provided meaningful insights into drug effectiveness and patient satisfaction trends. These results confirm that NLP-based sentiment analysis is effective in extracting valuable information from unstructured clinical reviews.

7.4 Impact of Sentiment Integration on Drug Ranking

The integration of sentiment scores into the drug ranking mechanism improved transparency and contextual awareness.

Drugs with high clinical prediction scores but low sentiment scores were ranked slightly lower, preventing potential recommendation of drugs with negative patient experiences.

The weighted scoring formula allowed adjustable influence of sentiment on final ranking. Experimental evaluation showed that moderate weighting achieved optimal balance between clinical accuracy and patient feedback.

This integration enhanced trustworthiness and supported patient-centered healthcare analytics.

7.5 Comparative Analysis with Traditional Systems

Compared to traditional rule-based systems, the proposed system demonstrated higher adaptability and personalization. Rule-based systems rely solely on clinical guidelines, while the proposed system incorporates real- world feedback through sentiment analysis. The hybrid approach provided improved contextual insights without compromising predictive accuracy. Quantitative comparison showed improved recommendation relevance and qualitative feedback suggested enhanced transparency. These findings highlight the advantages of integrating machine learning and NLP in healthcare recommendation systems.

7.6 User Interpretation and Transparency

One of the key advantages of the system is interpretability. The output interface displayed not only recommended drugs but also associated sentiment distribution. Healthcare professionals could observe both prediction probability and public sentiment trends before making decisions. This transparency increases confidence in automated decision-support systems and reduces reliance on black-box models. The interpretability feature enhances practical usability in clinical environments.

7.7 Limitations and Observed Challenges

Despite promising results, certain limitations were identified. The system relies heavily on dataset quality and diversity. Limited representation of rare diseases may affect prediction reliability. Sentiment analysis accuracy depends on preprocessing quality and domain-specific language understanding. Multi-drug interactions and dosage variations are not currently incorporated into the recommendation model. Future research may focus on incorporating adverse drug reaction prediction and multi-label classification.

7.8 Future Scope and Overall Discussion

Future improvements of the proposed system can focus on enhancing both its analytical capabilities and real-world applicability. One significant direction is the adoption of **deep learning–based sentiment analysis using transformer models** such as BERT or RoBERTa. These models provide superior contextual understanding compared to traditional NLP techniques, enabling more accurate interpretation of complex clinical reviews, sarcasm, and domain-specific language. This would lead to more reliable sentiment insights and a better understanding of patient experiences.

Another important enhancement is the **integration with real-time Electronic Health Record (EHR) systems**. By connecting the application with hospital databases and clinical information systems, the model can access up-to-date patient data, laboratory results, and treatment histories. This would significantly improve the clinical relevance and usability of the system, allowing it to function as a real-time decision-support tool within healthcare environments.

Additionally, the system can be extended to support **multi-condition patient profiles**, where

patients often present with multiple coexisting diseases.

Incorporating advanced logic for handling such complex cases, along with **drug–drug interaction analysis**, can greatly enhance the safety and accuracy of recommendations. This would help prevent adverse drug reactions and ensure that prescribed medications are compatible with the patient's existing treatment plan.

Further improvements may include the use of **explainable AI (XAI) techniques** to provide transparency in model predictions, enabling clinicians to understand the reasoning behind each recommendation. The inclusion of **personalized dosage prediction**, **risk assessment modules**, and **visual analytics dashboards** can also improve the overall effectiveness and user experience of the system.

Overall, the results of this project demonstrate that combining **patient condition-aware drug prediction with NLP-based sentiment analysis** significantly enhances healthcare decision-support systems. By integrating structured clinical data analysis with insights derived from unstructured patient reviews, the system offers a more comprehensive and balanced perspective for decision-making.

CHAPTER - 8
CONCLUSION

The project titled **"Patient Condition–Aware Drug Recommendation and Clinical Review Sentiment Analysis Using NLP and Machine Learning"** successfully demonstrates the integration of structured clinical data analysis with unstructured text mining to enhance healthcare decision-support systems. The primary objective of this research was to design and implement a machine learning-based framework capable of recommending appropriate drugs based on patient conditions while simultaneously analyzing patient-generated clinical reviews to extract sentiment insights. The results indicate that combining predictive modeling with sentiment analysis significantly improves transparency, personalization, and reliability in healthcare recommendations.

Traditional drug recommendation systems primarily rely on rule-based clinical guidelines or physician expertise. While these approaches are medically sound, they often lack adaptability and do not incorporate real-world patient feedback. Moreover, existing systems usually treat structured clinical data and unstructured review data separately. This separation limits the ability to integrate experiential insights into clinical decision-making. The proposed system addresses this limitation by bridging structured and unstructured healthcare data through machine learning and Natural Language Processing (NLP) techniques.

The system consists of two major components: a patient condition-aware drug recommendation module and a clinical review sentiment analysis module. The recommendation module uses supervised machine learning algorithms such as Random Forest or Logistic Regression to classify patient attributes and predict suitable medications. Structured patient features, including age, diagnosis, symptoms, and medical history, are transformed into numerical representations and used as input for model training. Cross validation and hyperparameter tuning ensure robust generalization and minimize overfitting.

The sentiment analysis module processes unstructured drug review text using NLP techniques. Preprocessing steps such as tokenization, stop word removal, and TF-IDF vectorization convert text into numerical features suitable for classification. Machine learning classifiers then categorize reviews into positive, negative, or neutral sentiments. Aggregated sentiment scores provide a summarized perception of each drug's effectiveness and tolerability from the patient's perspective.

The integration of both modules forms the core innovation of this project. Instead of relying solely on clinical prediction probabilities, the system combines prediction scores with sentiment scores using a weighted ranking mechanism. This ensures that recommended drugs are not only clinically suitable but also positively perceived by patients. The weighted integration formula allows flexible adjustment of clinical and experiential influence based on healthcare priorities.

Experimental results demonstrate that the drug recommendation model achieved high classification accuracy and balanced precision and recall values. Random Forest algorithms performed particularly well due to their ability to capture nonlinear relationships between patient attributes and drug categories.

Cross-validation confirmed stable performance across multiple data splits, indicating strong generalization capability.

The sentiment analysis model also achieved satisfactory accuracy and F1-scores. Logistic Regression provided efficient and reliable classification results. Although minor misclassifications occurred in ambiguous or complex reviews, overall performance remained consistent. Aggregated sentiment insights enhanced interpretability and provided additional confidence in drug selection decisions.

One of the key strengths of this system is transparency. Unlike traditional black-box models, the proposed framework provides clear insights into both clinical prediction scores and patient sentiment distributions. Healthcare professionals can evaluate not only which drug is recommended but also how it is perceived by other patients. This dual perspective enhances trust and supports evidence-based decision-making.

The system also demonstrates scalability and computational efficiency. Both recommendation and sentiment modules operate effectively on moderate hardware configurations. The modular architecture allows future integration with electronic health record systems or cloud-based healthcare platforms. Deployment through web frameworks further enhances accessibility and usability.

Despite its strengths, the project acknowledges certain limitations. The performance of the drug recommendation model depends heavily on dataset quality and diversity. Limited representation of rare medical conditions may affect predictive reliability. Additionally, sentiment analysis accuracy is influenced by domain-specific terminology and language complexity in medical reviews. Sarcasm or ambiguous expressions may challenge classification accuracy.

Another limitation is the absence of multi-drug interaction analysis and dosage considerations. The current model focuses primarily on single-drug classification based on patient conditions. Expanding the system to include adverse drug interaction detection would further enhance safety and reliability.

Future research directions include incorporating deep learning-based NLP models such as transformer architectures to improve contextual understanding of medical reviews. Integrating multi-label classification techniques can support patients with multiple concurrent conditions.

Real-time integration with hospital EHR systems can further enhance clinical applicability. Additionally, incorporating explainable AI techniques may improve interpretability and regulatory compliance.

From an academic perspective, this project contributes to intelligent healthcare analytics by integrating supervised machine learning and NLP techniques into a unified framework.

It highlights the importance of combining structured clinical analytics with unstructured patient feedback to create comprehensive decision-support systems. The research demonstrates how interdisciplinary approaches involving data science, healthcare, and natural language processing can address real-world challenges.

In conclusion, the Patient Condition–Aware Drug Recommendation and Clinical Review Sentiment Analysis System achieves its primary objectives effectively. The integration of machine learning-based clinical prediction with NLP-driven sentiment analysis enhances personalization, transparency, and reliability in drug recommendation systems. The project successfully bridges structured and unstructured healthcare data, contributing to advanced decision-support technologies in modern healthcare environments.

The results validate the potential of machine learning and NLP in improving healthcare analytics and patient-centered care. With further enhancements and integration into clinical systems, such intelligent frameworks have the potential to significantly improve treatment outcomes and healthcare decision-making processes.

REFERENCES

1. *T. Jiang et al., "Digital health interventions from the humanistic perspective of sports: Strategies to promote health for all," Frontiers in Public Health, vol. 14, p. 1780308, 2026.*
2. *P. Patange Aparna and C. Rokde, "Artificial intelligence and machine learning in neurology," Artificial Intelligence and Machine Learning in Neurology, vol. 2, pp. 785–809, 2026.*
3. *M. Atta, A. Ragab, M. A. Salam, and M. Mahmoud, "Signal and image processing applications in healthcare," International Journal of Advances in Signal and Image Sciences, pp. 1870–1879, 2026.*
4. *L. Wreyford et al., "Applications of digital health technologies in nursing practice," Nursing Reports, vol. 16, no. 1, p. 18, 2026.*
5. *R. Bounaama and M. E. A. Abderrahim, "Medical engineering and informatics approaches for healthcare systems," International Journal of Medical Engineering and Informatics, vol. 18, no. 1, pp. 25–35, 2026.*
6. *R. Kumar et al., "Biomedical signal processing for intelligent healthcare systems," Biomedical Signal Processing and Control, vol. 119, p. 109742, 2026.*
7. *M. S. Aslam and M. Jamal, "Emerging trends in science and technology for healthcare," Global Trends in Science and Technology, vol. 2, no. 2, pp. 23–42, 2026.*
8. *S. Phadtare, N. Kulkarni, and M. Devare, "Network modeling in health informatics and bioinformatics," Network Modeling Analysis in Health Informatics and Bioinformatics, vol. 15, no. 1, p. 54, 2026.*

9. *Mukhtar et al., "Artificial intelligence applications in Parkinson's disease," Parkinsonism & Related Disorders, 2026.*

10. *M. Jaradat and M. Awad, "Artificial intelligence in medical informatics," International Journal of Medical Informatics, 2026.*

11. *K. Yang et al., "Artificial intelligence for healthcare analytics," arXiv preprint arXiv:2510.16708, 2025.*

12. *N. Rathnasekara and U. Wijenayake, "ICT applications in emerging healthcare systems," International Journal on Advances in ICT for Emerging Regions, vol. 18, no. 2, 2025.*

13. *S. Kumar et al., "Machine learning approaches in autonomous systems," in Proc. Int. Conf. Machine Learning and Autonomous Systems (ICMLAS), pp. 1516–1522, 2025.*

14. *Srinivas, P. H. Vardhan, and K. V. Reddy, "Applications of computer science in modern systems," International Journal of Advanced Research in Computer Science, vol. 16, no. 2, 2025.*

15. *P. Saxena and J. Devashish, "Computational intelligence techniques," in Proc. Int. Conf. Computational, Communication and Information Technology (ICCCIT), pp. 892–897, 2025.*

16. *S. K. Hegde, R. Hegde, and T. Murugan, "Advancements in computation and computer technologies," in Proc. Int. Conf. Advancement in Computation & Computer Technologies (InCACCT), pp. 189–194, 2025.*

17. *S. Golder et al., "Use of social media data in drug safety monitoring," Drug Safety, vol. 48, no. 4, pp. 321–337, 2025.*

18. *J. Ye et al., "Clinical data analysis using machine learning," medRxiv, 2024.*

19. *K. P. Swain et al., "Bioinformatics approaches in healthcare," The Open Bioinformatics Journal, vol. 17, no. 1, 2024.*

20. *H. Zheng et al., "Innovations in science and technology," Academic Journal of Science and Technology, vol. 10, no. 1, pp. 62–68, 2024.*

21. *Anbazhagan, E. Sophiya, and R. P. Kumar, "IT applications in healthcare systems," International Journal of Information Technology, vol. 16, no. 8, pp. 5195–5212, 2024.*

22. *U. Ahmed et al., "Intelligent connected systems," Journal of Intelligent Connected Emerging Technologies, vol. 8, no. 2, pp. 1–17, 2023.*

23. *S. K. Nayak et al., "Machine learning applications in healthcare," IEEE Access, vol. 11, pp. 99304–99318, 2023.*

24. *S. A. Alsaif et al., "NLP-based bi-directional recommendation system: Towards recommending jobs to job seekers and resumes to recruiters," Big Data and Cognitive Computing, vol. 6, no. 4, p. 147, 2022.*

25. *U. Bhimavarapu, N. Chintalapudi, and G. Battineni, "A fair and safe usage drug recommendation system in medical emergencies by a stacked ANN," Algorithms, vol. 15, no. 6, p. 186, 2022.*